# Spanner

## Software Requirements Specification v2.2

incorporating the Feasibility Study, Risk Register and System Architecture & Design

ITC327W Assessment 2 - Phase 2: System Architecture and Design (Unit 3)

Riese Auto-Clinic - Stakeholder: Michael Riese, Owner/Head Mechanic
GitHub Repository: https://github.com/BlackZ3ro/Spanner-Riese-Auto-Clinic


| **Group Y ("The Zealots")** | |
|---|---|
| **Student Name** | **Student Number** |
| Wisani Nkuna | 222010787 |
| Gary van Wyk | 222048911 |
| Keamogetse Ogone Kgosi (Kea) | 222003092 |
| Moremi KE (Khumo) | 222062729 |
| Mathye MD (Denny) | 221014861 |
| Ofentse Rammutla (joined 17 Sept 2026) | 223056856 |
| Breyton Rossouw (joined 17 Sept 2026) | 222069744 |


Version 2.2 - compiled 11 October 2026

# Revision History and How to Read This Document

| Version | Date | Phase | Summary |
|---|---|---|---|
| 1.0 | 4 Sept 2026 | Phase 1 (Units 1-2) | SRS, Feasibility Study and Risk Register as submitted and assessed (80%). |
| 2.0 | 30 Sept 2026 | Phase 2 (Unit 3) | Adds Part B: requirement and scope updates (Changes 7-11), system architecture, use case, sequence and entity relationship diagrams, interface designs, design alignment, risk register updates and links to the editable design sources. |
| 2.1 | 7 Oct 2026 | Phase 2 (Unit 3) | Adds the navigation flow, the customer tracking screens for both apps (web page built; mobile screen designed and compiled), cleaner architecture and ERD figures, the editable draw.io source for every design, a database security finding (R-17) and the Microsoft Project section. |
| 2.2 | 11 Oct 2026 | Phase 2 (Unit 3) | Adds the team's interface designs (Gary's Flutter mockups, Denny's web designs), the client's colour choice (gold, green and orange) applied to the web app, the updated Microsoft Project schedule (Fig. 16) and the final list of open items. |

The brief asks for **one updated SRS** rather than a separate design document. This version therefore keeps the Phase 1 content exactly as it was submitted and assessed (**Part A**, pages that follow, unchanged apart from this new cover) and adds the Phase 2 work as **Part B**. Where Phase 2 changes a Phase 1 statement, Part B says so explicitly and references the Phase 1 section, so the history of each decision stays visible instead of being overwritten.

### Changes in Part B that amend Part A

| Part A reference | Amended by | Effect |
|---|---|---|
| 1.7 Functional Requirements | Section 4.1 | New FR-14 to FR-18. |
| 1.10 Requirements Change Log | Section 4.2 | New Changes 7-11. **Change 7 partially reverses Change 5** (customer-facing status). |
| 1.11 Refined Project Scope (Out of Scope) | Section 4.3 | A coarse, login-free repair-status lookup moves into scope; live/granular tracking and ETAs stay out. |
| 3. Project Risk Register | Section 4.4 | R-03 closed, R-10 materialised, new R-14 to R-17. |
| 1.2 Affected users | Section 5.2 | Customers become a (limited, login-free) direct user of one screen. |

> **Draft status.** Items still owed by named members before submission on 12 October 2026 are listed in Section 6.3 and marked "Pending" where they appear.

*Compiled from group chat content, the group's shared Google Doc, and the Assessment 1 brief. Amber "Assumption/flag" notes mark places where a role or detail was inferred rather than confirmed by the group; please correct these before submission. The Microsoft Project schedule is submitted as a separate file (see Appendices) and is referenced only at a high level here.*

*Updated per consultation feedback from Ms Mbele (Aug 2026): document justified throughout in Arial with 1.15 line spacing, dedicated cover page added, directly/indirectly affected users clarified (Section 1.2), and the suggestion of customer-facing real-time job tracking evaluated and deliberately excluded from scope (Section 1.10, Change 5; Section 1.11).*

# 1. Software Requirements Specification

## 1.1 Project Introduction

Spanner (working title) is a workshop management system for Riese Auto-Clinic, a mechanic shop, combining a Flutter mobile application and an ASP.NET web application on a shared Supabase backend. It allows shop staff to manage parts/stock, book in vehicles and assign jobs, capture labour, and generate customer-ready PDF invoices, replacing the shop's current manual, paper- and memory-based process with a single, synchronised system usable from the workshop floor (mobile) or the office (web).

## 1.2 Stakeholder Profile and Current Process

Organisation: Riese Auto-Clinic, a mechanic shop. Direct stakeholder: Michael Riese, Owner/Head Mechanic, selected for his direct authority over how the shop operates and his ability to give first-hand input into system requirements.

The group's real name and business name are used throughout this document with the stakeholder's knowledge and consent, given the group's direct working relationship with him; this was confirmed as acceptable rather than substituting an anonymised company name.

### Directly and Indirectly Affected Users

Following clarification obtained during academic consultation, affected users are classified below by whether they operate the system themselves or are only affected by what it produces:

- Directly affected: Michael Riese (Owner/Head Mechanic), with overall oversight of bookings, stock and invoicing, and may also perform mechanic and front-desk duties personally.
- Directly affected: Mechanics, who log labour time against a job, view assigned jobs and vehicle details, and check parts availability.
- Directly affected: Front-desk/admin staff, who book in vehicles, record parts and stock, generate and hand over invoices, and are typically the ones speaking to a customer during drop-off, consultation and collection.
- Indirectly affected: Shop customers, who do not log into or operate the system themselves, but are affected by what it produces, namely the invoice/quote and any calls or messages staff choose to send them.

Current process: vehicles are booked in verbally, through direct communication with admin staff or the owner. Once a vehicle arrives, mechanics begin diagnostics immediately to identify faults or service points. When parts are needed, a trip is made to a local supplier to source

them, sometimes across multiple stores if one does not stock everything required, before returning to install them and notify the customer of completion or complications. Once work is finished, payment is calculated by combining parts cost, a markup, and labour time, and the job is signed off. The shop keeps customer contact details along with vehicle-identifying information (odometer reading, licence plate and VIN) specifically to protect the business in case a customer becomes non-compliant.

Evidence: informal stakeholder discussion conducted directly with Michael Riese (a personal contact of a group member), documented in the group's shared planning document. A formal, scheduled stakeholder interview (Unit 1 Activity 1.2) confirmed and extended this record; see Section 1.6 and the Requirements Change Log.

## 1.3 Problem Statement

Current situation: bookings, parts sourcing, labour tracking and invoicing are all handled manually, relying on verbal communication, physical trips to suppliers and the owner's memory or paper notes for cost and stock information.

Problem: manually booking in a vehicle and then sourcing parts from multiple locations, when a single supplier does not carry everything needed, creates unnecessary time delays on jobs. There is no system to track where components were bought for future reference, how much stock of particular parts is held, or labour time spent per vehicle in a structured way.

Consequences: jobs take longer to complete than necessary, restocking decisions are made without good information, and producing an accurate, professional invoice at the end of a job is slower and more error-prone than it needs to be.

Need for improvement: the shop needs a system that documents each job as it comes in, tracks where parts are sourced for future purchasing decisions, maintains a record of stocked parts/service kits, tracks labour time per vehicle, and produces a clear invoice itemising component costs and labour at the end of the job.

## 1.4 Project Aim and Objectives

Aim: to design and develop an integrated mobile and web-based workshop management system for Riese Auto-Clinic that allows the business to manage parts/stock, book in vehicles and assign jobs, capture labour, and generate customer-ready invoices from a single, synchronised platform.

Objectives:

- Enable staff to record and manage parts/stock inventory, including where a part was bought, its cost price, and the markup/sell price charged to the customer.
- Allow staff to book in a vehicle (capturing customer contact details, VIN, licence plate and odometer reading) and assign one or more jobs to it (e.g. service, engine, suspension).
- Allow staff to capture labour hours worked on a job, together with an hourly labour rate, once the job is marked as finished.
- Automatically calculate the total job cost (parts + labour) when a job is completed.
- Generate a PDF invoice/quote for the completed job that can be sent to the customer via WhatsApp or email, or printed.

- Synchronise data in real time between the Flutter mobile app and the ASP.NET web app using a shared Supabase backend.
- Keep parts sourcing information (supplier, cost) on record to support future purchasing decisions and reduce delays from re-sourcing parts.
- Display a reference image of the vehicle, looked up from brand, model and optional year, so mechanics have a quick visual on what they are working with when a job comes in by phone or message.

## 1.5 Project Scope (initial)

In scope: a Flutter mobile application and an ASP.NET web application sharing one Supabase backend; parts/stock management (source, cost price, sell price/markup); vehicle booking and job assignment; labour hour and rate capture; automatic job costing; PDF invoice/quote generation shareable via WhatsApp, email or print.

Out of scope: online payment processing; automated customer appointment reminders; support for multiple branches/franchise locations; a customer-facing login portal.

Acceptance criteria: both the mobile and web versions must clearly log the work done on a job and be able to export a PDF invoice for it.

Constraints: the project must be completed within a single semester, with no monetary cost involved in developing it.

Assumptions: the stakeholder (Michael Riese) will remain available for feedback and confirmation throughout the project.

## 1.6 Stakeholder Needs

The following needs were confirmed directly with Michael Riese through the formal stakeholder interview conducted for Activity 1.2 (transcript below): a combined mobile and web system that keeps track of stock, where parts were sourced and their costs; the same stock visible to all mechanics and to admin, so mechanics know what's available and admin can track money spent per job; a central system linking parts and labour to the specific job and vehicle they relate to, rather than tracking them separately; each mechanic able to log their own labour time against a job, so admin has a clear record of who did what and for how long when preparing an invoice; and an invoice generated automatically once a job is finished, including both labour and parts cost.

### Stakeholder Interview Transcript (Activity 1.2)

*Conducted 19 August 2026. Present: Gary van Wyk (group) and Michael Riese (stakeholder).*

**Q1** So you would like us to make your life a bit easier, how can we do that for you?

A1: We would like an app for mobile and a desktop/web version that can keep track of all the stock we keep, where we got the parts, and their costs.

**Q2** Okay, that makes sense. And I would assume you would want the app and web version to talk to each other and keep track of the same stock and things?

A2: Yes. We'd like all the mechanics to be able to see what we have available, what needs to get bought, and so on, but also so that the admin personnel can also keep track of the money spent on each job that comes in.

**Q3** Alright, so we need to develop a back end database that tracks parts, and for which jobs they're being used for. Might I also make a suggestion? If we are tracking the parts, why not track what types of jobs is being performed on different vehicles? Have a central tracking/management system?

A3: That would be perfect. Having admin and the mechanics all stay in the loop and on the same page with what is being done and the costs involved with it as well.

**Q4** That is what I had in mind as well. We can even make it so that every mechanic working on a vehicle is able to log their labour time on the app, admin can see it, and when developing the invoice, have a clear track of who did what on which vehicle and for how long, if that is what would help as well?

A4: That would honestly help us a lot with paperwork and unnecessary admin work.

**Q5** Is there anything else we can do for you or that we can include into this system that would improve your situation?

A5: Yes, actually, if it would be possible, to have the system itself export an invoice after all the work is done on a vehicle, which includes all the labour and cost price.

*Assumption/flag: The signed stakeholder confirmation letter referenced as evidence for this interview has been collected, countersigned by Michael Riese, and is reproduced in full as Appendix E.*

*Assumption/flag: The information above (current process, costs and figures) reflects what Michael Riese described directly in discussion and interview; it is not simulated/dummy data. Where specific figures are not yet known (e.g. exact markup percentages, average job values), this will be updated with the shop's actual transaction data as it becomes available, rather than left as a placeholder or generic estimate.*

## 1.7 Functional Requirements

**FR-01** The system shall allow staff to register a new vehicle booking with customer contact details, VIN, licence plate and odometer reading.

**FR-02** The system shall allow staff to assign one or more job types (e.g. service, engine, suspension) to a booked vehicle.

**FR-03** The system shall allow staff to record parts/stock items, including supplier, purchase (cost) price and sell price/markup.

**FR-04** The system shall allow staff to link parts used to a specific job on a specific vehicle.

**FR-05** The system shall allow staff to capture labour hours worked and an hourly labour rate for a job.

**FR-06** The system shall automatically calculate the total job cost (parts + labour) once a job is marked complete.

**FR-07** The system shall generate a PDF invoice/quote summarising parts, labour and total cost for a completed job.

**FR-08** The system shall allow a generated invoice PDF to be shared via WhatsApp or email, or printed.

**FR-09** The system shall synchronise stock, booking and job data between the mobile app and web app in real time via Supabase.

**FR-10** The system shall allow authorised staff to view current stock levels of parts/service kits.

**FR-11** The system shall maintain a per-vehicle history of past jobs, searchable by VIN or licence plate.

**FR-12** The system shall restrict access to shop data to authenticated staff accounts only.

**FR-13** The system shall look up and display a reference image of the vehicle from Wikimedia, based on the brand, model and (optionally) year captured at booking, as a best-effort visual aid rather than authoritative data.

## 1.8 Non-Functional Requirements

**NFR-01 (Usability)** The interface must be simple enough for mechanics with limited software experience to use with minimal training.

**NFR-02 (Performance)** Core actions (booking a vehicle, adding a part, generating an invoice) should complete in a few seconds under normal connectivity.

**NFR-03 (Availability)** The mobile app should tolerate brief connectivity loss without losing unsaved job data, given real connectivity gaps already experienced by the group.

**NFR-04 (Security)** Customer and vehicle data must be protected via authenticated access and Supabase Row Level Security, and must never be committed to the public GitHub repository.

**NFR-05 (Data integrity)** Financial calculations (cost, markup, totals) must be accurate and auditable.

**NFR-06 (Compatibility)** The mobile app must run on common Android/iOS devices used by shop staff.

**NFR-07 (Maintainability)** The codebase must be organised and documented in GitHub to support future extension beyond this project.

## 1.9 Integration Requirements

The Flutter mobile app and the ASP.NET web app both connect to a single shared Supabase project (PostgreSQL database, Auth, Storage and Realtime) as the single source of truth for stock, bookings, jobs and invoices. Authentication is handled through Supabase Auth and shared across both clients so staff use one account regardless of which app they open. Changes made on one client (e.g. a new stock entry from the mobile app) propagate to the other (e.g. the web dashboard) via Supabase Realtime subscriptions. PDF invoice generation is triggered on job completion and shared through the device's native share sheet (mobile) or a direct download/share action (web), avoiding the cost and complexity of a dedicated messaging API.

## 1.10 Requirements Change Log

| Change | What Changed | Reason |
|---|---|---|
| 1 | Web application platform specified as ASP.NET rather than a generic web app. | Aligned to the ASP.NET + Supabase technical requirement set out in the Assessment 1 brief. |
| 2 | Customer-facing login portal removed from scope. | Operational feasibility: the direct users are shop staff, not customers; economic/schedule feasibility favoured a tighter scope. |
| 3 | WhatsApp/email sharing implemented via native device share sheet rather than a paid messaging API. | Economic feasibility: keeps the project at zero development cost and avoids third-party API complexity. |
| 4 | Multi-branch/franchise support explicitly excluded. | Riese Auto-Clinic is a single-location business; out of scope was confirmed with the stakeholder relationship in |

| Change | What Changed | Reason |
|---|---|---|
| | | mind. |
| 5 | Customer-facing real-time job/status tracking explicitly excluded from scope (both mobile and web remain staff-only tools). | Suggested as a possible enhancement during academic consultation, then evaluated directly against stakeholder operational experience (Michael Riese, via Gary). Giving customers a granular status or ETA creates false expectations when parts-sourcing delays are common and outside the shop's control, and would require mechanics to manually log fine-grained progress updates with no operational benefit. All customer-facing output stays limited to the generated invoice/quote and direct calls or messages from staff, matching how the shop already operates. |
| 6 | Added a vehicle reference image lookup (FR-13): a Wikimedia-based scraper takes the brand, model and optional year captured at booking and returns a closest-match reference image. | Proposed by Gary as a low-cost visual aid so staff instantly recognise a vehicle from a phone call or message. Uses the free Wikimedia API rather than a paid image service, matching the zero-budget constraint. The scraper is not 100% accurate; better accuracy would require a paid API, which is why the feature is a supplementary visual aid (see R-13) rather than a core requirement. |

## 1.11 Refined Project Scope

Following the stakeholder interview above and the feasibility and risk analysis in Sections 2 and 3, the group finalised the scope below to carry forward into system design.

**Project Title:** Spanner (RT)

**Project Aim:** Develop an integrated mobile and web system that allows Riese Auto-Clinic to book vehicles, assign and track jobs, manage parts stock and purchase history, log labour time per mechanic, and generate detailed invoices, all from a shared central system.

### Objectives

- Provide mechanics with a simple way to view available stock, log labour time, and record parts used on jobs.
- Allow admin staff to track costs, monitor job progress, and manage stock levels.
- Maintain a single shared source of truth for jobs, parts, labour and costs across both the mobile and web applications.
- Reduce paperwork and administrative effort by automatically generating clear invoices.
- Give staff a quick visual reference of the vehicle via a Wikimedia-based image lookup.

### In Scope

- Vehicle booking and job creation.
- Assignment of jobs to mechanics.
- Logging of labour time per mechanic per job.
- Tracking of parts on each job (including supplier and cost).
- Basic stock management (quantity on hand and low-stock awareness).
- Recording of the type of work performed.
- Generation and export of a detailed invoice (PDF) showing parts costs and labour.
- Shared data between the Flutter mobile app and ASP.NET web application via Supabase.

- Vehicle reference image lookup by brand, model and optional year (Wikimedia, best-effort accuracy).

### Out of Scope

- Customer-facing portal or client login.
- Customer-facing real-time job/status tracking: raised as a suggestion during academic consultation but deliberately excluded per direct stakeholder operational preference (see Requirements Change Log, Change 5).
- Online payment processing.
- Advanced inventory forecasting or automatic reordering.
- Integration with external accounting software.
- Real-time vehicle tracking.
- Support for multiple workshop branches.

### Deliverables

- Flutter mobile application.
- ASP.NET web application.
- Shared Supabase backend and database.
- Ability to export a clear invoice PDF.

### Constraints

- Must be completed within a single semester.
- No monetary costs for development tools or services (free tiers only).
- Prototype quality is acceptable (not a fully production-ready system).

### Assumptions

- The stakeholder (Michael Riese) and relevant staff will remain available for feedback and meetings throughout the project.
- Mechanics and admin staff will have access to smartphones and computers.
- Basic internet connectivity will be available at the workshop.

## 2. Feasibility Study

| Area | Evaluation |
|---|---|
| Technical | Flutter (mobile) and Supabase (backend: Postgres, Auth, Storage, Realtime) are already in use, with a working Supabase project and an in-progress mobile prototype giving the group real technical traction. The web client will use ASP.NET, integrating with the same Supabase backend. The main technical gap is the group's limited prior ASP.NET experience, which is a manageable learning curve given the semester timeframe and Supabase handling most backend logic. PDF generation and native share-to-WhatsApp/email are both achievable with free, well-documented libraries. A vehicle reference image lookup (Gary) scrapes Wikimedia for a closest-match image based on brand, model and optional year, giving mechanics a quick visual reference; it is a supplementary feature layered on top of the core requirements, not a dependency for them. Conclusion: technically feasible, with ASP.NET familiarisation flagged as a schedule risk (see Risk Register). |
| Operational | The proposed workflow (book a vehicle, assign a job, capture parts and labour, generate an invoice) maps directly onto Riese Auto-Clinic's actual process as described by Michael Riese, so no retraining in a new way of working is required; only digitising steps staff already perform. The main operational requirement is a simple, low-friction interface, since the direct users are |

| Area | Evaluation |
|---|---|
| | mechanics and admin staff rather than technical staff. Conclusion: operationally feasible provided the interface stays simple (see NFR-01). |
| Economic | The project uses free-tier tooling throughout: Flutter and ASP.NET are free and open-source, Supabase's free tier covers a project of this scale, GitHub is free for the group's use, and the vehicle reference image lookup uses the free Wikimedia API rather than a paid image service. Sharing invoices uses the device's native share sheet rather than a paid WhatsApp Business API or SMS gateway. This matches the stated project constraint of zero development budget. Conclusion: economically feasible at no direct cost to the group or the stakeholder. |
| Schedule | The group has a head start (mobile prototype and Supabase project already created before the module formally began) and a single-semester deadline. The realistic risk is the web application (ASP.NET) and full mobile-web-Supabase integration, which have not yet started. Conclusion: feasible only if the remaining scope is kept tight, as the refined scope below deliberately excludes lower-priority features to protect the schedule. |
| Legal | The system stores personal customer contact details and vehicle-identifying data (VIN, licence plate, odometer) that Riese Auto-Clinic already collects for its own compliance protection. This data must be handled in line with South Africa's Protection of Personal Information Act (POPIA): collected only for the stated business purpose, access-restricted to authenticated staff via Supabase Row Level Security, and never committed to the public GitHub repository. Conclusion: legally feasible provided access control and data-minimisation practices in NFR-04 are implemented. |
| Ethical | The system must represent parts costs and labour charges to customers accurately and transparently, since the invoice is the customer-facing record of what they are charged. Stakeholder and customer information must only be used for the stated project and assessment purpose. Conclusion: ethically feasible with no material concerns identified, provided invoice calculations remain accurate and auditable (see NFR-05). |

## Overall Feasibility Decision

The project is feasible with changes. It is well supported technically and economically by the group's existing Supabase project and mobile prototype, and it is operationally and ethically sound because it digitises a process the stakeholder already follows. The two conditions attached to this decision are: (1) the ASP.NET web application must start in parallel with continued mobile work rather than after it, given the group's limited prior ASP.NET experience (see R-03), and (2) the scope must stay to the refined list in Section 1.11 to protect the single-semester schedule.

# 3. Project Risk Register

*Likelihood and Impact are rated Low / Medium / High (1–3); Score = Likelihood × Impact, bucketed as Low (1–2), Medium (3–4) or High (6–9).*

| ID | Risk Statement | Category | Likelihood | Impact | Score / Level | Mitigation | Contingency | Owner | Status |
|---|---|---|---|---|---|---|---|---|---|
| R-01 | Michael Riese's availability for feedback and confirmation reduces as the semester progresses, delaying requirement sign-off. | Stakeholder | Medium | High | 6 / High | Agree a light-touch, low-effort check-in channel (WhatsApp) early, and batch questions to minimise his time burden. | Fall back on previously confirmed requirements and document any assumption explicitly for later confirmation. | Gary | Open |
| R-02 | The stakeholder requests additional features once he sees the working prototype, expanding scope beyond what the semester allows. | Scope | Medium | Medium | 4 / Medium | Maintain a clearly refined, agreed scope (below) and log any new request in the requirements change log rather than building it immediately. | Park new requests as a 'Phase 2 / future work' list to revisit after submission. | Denny | Open |
| R-03 | The web application (ASP.NET) has not yet started and the group has limited prior ASP.NET experience, risking a late or rushed build. Ownership sitting with Gary alongside mobile and Supabase also concentrates technical work on one person. | Technical | Medium | High | 6 / High | Start ASP.NET scaffolding and the Supabase connection early in Phase 2, in parallel with mobile work rather than after it; consider pulling in Khumo or Kea for support if Gary's workload becomes a bottleneck. | Reduce the web app to a read-only dashboard of the same Supabase data if full parity can't be reached in time. | Gary | Open |
| R-04 | The project outgrows Supabase's free-tier storage/bandwidth limits before the project is demonstrated. | Technical / Infrastructure | Low | Medium | 2 / Low | Monitor usage via the Supabase dashboard monthly; avoid storing large files (e.g., full-resolution photos) unnecessarily. | Compress or offload large assets, or apply for Supabase's student/hobby tier if a limit is approached. | Gary | Open |
| R-05 | Real-time sync between the mobile app and web app produces conflicting or duplicate records (e.g., two staff editing the same job). | Technical / Integration | Low | Medium | 2 / Low | Use Supabase's row-level timestamps and design simple, single-writer-per-record flows where possible. | Add a manual 'last updated by' field so conflicts are visible and can be corrected by staff. | Gary | Open |
| R-06 | A group member loses connectivity or availability at short notice (already | Team / Resource | Medium | Medium | 4 / Medium | Keep the progress tracker current so gaps are visible early, and | Reassign or share the affected task temporarily among members with | Wisani | Monitoring |

| ID | Risk Statement | Category | Likelihood | Impact | Score / Level | Mitigation | Contingency | Owner | Status |
|---|---|---|---|---|---|---|---|---|---|
| | experienced once this phase), delaying their task. | | | | | maintain a WhatsApp-first coordination habit that doesn't depend on any one channel. | capacity. | | |
| R-07 | Task completion becomes uneven across members as deadlines approach, with some falling behind. | Team / Schedule | Medium | Medium | 4 / Medium | Review the Task Progress tracker weekly and flag slipping tasks before they become blockers. | Redistribute the task or bring in support from a member with lighter load (as already planned for Khumo supporting Gary). | Wisani | Monitoring |
| R-08 | The formal stakeholder interview (U1-05) and supporting evidence are not finalised in time to support the SRS. | Documentation / Evidence | Low | High | 3 / Medium | Schedule the interview as a fixed WhatsApp call this week rather than leaving it open-ended, given direct access to the stakeholder. | Use the existing informal stakeholder discussion as interim evidence, clearly logged as informal pending the formal record. | Gary | Open |
| R-09 | The GitHub repository is not set up in time, leaving no evidence of individual contribution as required by the assessment. | Project Management | Low | High | 3 / Medium | Kea has taken ownership of repository setup and should create it with a proper README promptly. | If delayed, migrate existing local project history into the repository as soon as it is created so contribution history is not lost. | Kea | Open |
| R-10 | Customer and vehicle data (contact details, VIN, licence plate) is exposed through inadequate access control or accidentally committed to GitHub. | Legal / Compliance | Low | High | 3 / Medium | Apply Supabase Row Level Security from the start and add real stakeholder data to .gitignore / exclude from any repository content. | Rotate any exposed credentials immediately and review repository history to remove the exposure if it occurs. | Denny | Open |
| R-11 | PDF invoice generation does not integrate cleanly with the chosen library, producing incorrectly formatted or incomplete invoices. | Technical | Low | Medium | 2 / Low | Prototype the PDF generation step early with sample data rather than leaving it until the end of development. | Fall back to a simpler, text-based PDF layout if a more polished template proves too time-consuming to implement. | Gary | Open |
| R-12 | The Microsoft Project schedule underestimates remaining effort, causing later phases to slip. | Schedule | Medium | Medium | 4 / Medium | Base task durations on actual Phase 1 pace rather than optimistic estimates, and review the schedule against real progress each | Cut lower-priority scope items (see refined scope) rather than compressing testing or integration time. | Denny | Open |

| ID | Risk Statement | Category | Likelihood | Impact | Score / Level | Mitigation | Contingency | Owner | Status |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | phase. | | | |
| R-13 | The Wikimedia-based vehicle image lookup returns an inaccurate or missing reference image, since the free scraper is not 100% accurate by design. | Technical | Medium | Low | 2 / Low | Treat the image as a best-effort visual aid only, never authoritative vehicle data; fall back to showing the brand/model/year as plain text if no match is found. | If accuracy proves too poor to be useful, disable the auto-image feature and keep vehicle identification text-only, or evaluate a paid image API in a future phase if budget allows. | Gary | Monitoring |

### Three Highest Project Risks

- R-01: Stakeholder availability declining over the semester (Score 6/High). Everything downstream depends on continued access to Michael Riese.
- R-03: ASP.NET web app not yet started, limited prior experience (Score 6/High). This is the single biggest technical gap between current progress and the full scope.
- R-12: Microsoft Project schedule underestimates remaining effort (Score 4/Medium). This is the next-highest risk, and directly relevant once the current draft schedule is reviewed against actual Phase 1 pace.

*Assumption/flag: R-03 (ASP.NET web app) is confirmed to Gary, who owns the mobile app and Supabase setup and will also build the web app. R-09 (GitHub setup) is confirmed to Kea, who volunteered for it directly. R-13 (vehicle image lookup accuracy) is a new, low-severity risk added alongside the feature itself; it is being monitored rather than treated as a blocker.*

## Appendices

Appendix A: GitHub Repository. https://github.com/BlackZ3ro/Spanner-Riese-Auto-Clinic. Contains the group's source code, folder structure and commit history as evidence of individual contribution.

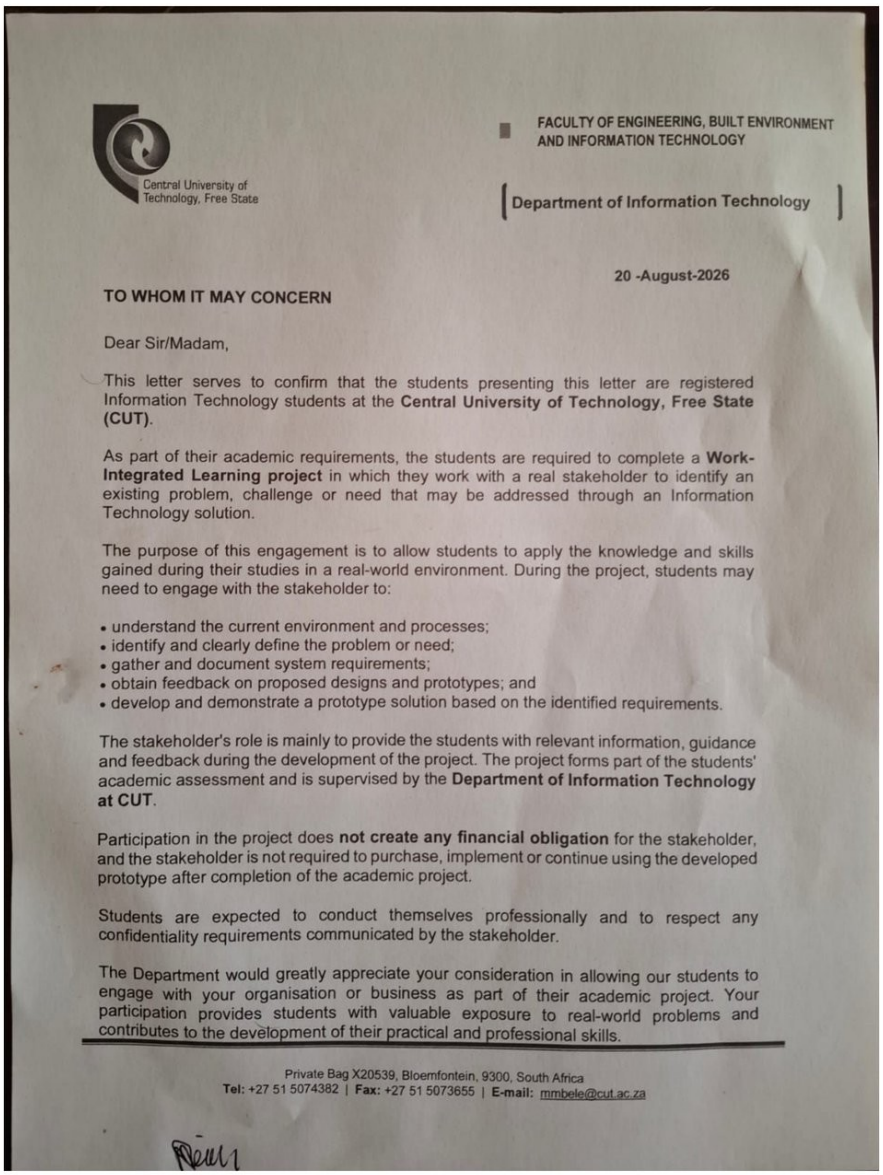
Appendix B: Stakeholder Interview Transcript. Included in full in Section 1.6 above (Activity 1.2, conducted 19 August 2026 with Gary van Wyk and Michael Riese).

Appendix C: Academic Consultation Notes. Feedback received from Ms Mbele (module mentor) on this document was reviewed and applied throughout; see the formatting and content notes referenced in Sections 1.2, 1.6, 1.10 and 1.11.

Appendix D: Microsoft Project Schedule. Submitted as a separate file (TheZealotsSchedule.mpp) alongside this document and the Progress Checker, per the submission instructions; not reproduced here as it is not a static document.

Appendix E: Signed Stakeholder and Departmental Confirmation Letter. Reproduced in full overleaf. Confirms the group's Work-Integrated Learning engagement with the Department of Information Technology at CUT and with the stakeholder, Michael Riese of Riese Auto-Clinic, and carries both the department's digital signature (Mpho Mbele, Departmental Manager, 20 August 2026) and Michael Riese's handwritten signature.

# Appendix E: Signed Stakeholder and Departmental Confirmation Letter



Central University of Technology, Free State

FACULTY OF ENGINEERING, BUILT ENVIRONMENT AND INFORMATION TECHNOLOGY

Department of Information Technology

20 -August-2026

**TO WHOM IT MAY CONCERN**

Dear Sir/Madam,

This letter serves to confirm that the students presenting this letter are registered Information Technology students at the **Central University of Technology, Free State (CUT)**.

As part of their academic requirements, the students are required to complete a **Work-Integrated Learning project** in which they work with a real stakeholder to identify an existing problem, challenge or need that may be addressed through an Information Technology solution.

The purpose of this engagement is to allow students to apply the knowledge and skills gained during their studies in a real-world environment. During the project, students may need to engage with the stakeholder to:

- understand the current environment and processes;
- identify and clearly define the problem or need;
- gather and document system requirements;
- obtain feedback on proposed designs and prototypes; and
- develop and demonstrate a prototype solution based on the identified requirements.

The stakeholder's role is mainly to provide the students with relevant information, guidance and feedback during the development of the project. The project forms part of the students' academic assessment and is supervised by the **Department of Information Technology at CUT**.

Participation in the project does **not create any financial obligation** for the stakeholder, and the stakeholder is not required to purchase, implement or continue using the developed prototype after completion of the academic project.

Students are expected to conduct themselves professionally and to respect any confidentiality requirements communicated by the stakeholder.

The Department would greatly appreciate your consideration in allowing our students to engage with your organisation or business as part of their academic project. Your participation provides students with valuable exposure to real-world problems and contributes to the development of their practical and professional skills.

Private Bag X20539, Bloemfontein, 9300, South Africa
**Tel:** +27 51 5074382 | **Fax:** +27 51 5073655 | **E-mail:** mmbele@cut.ac.za

Central University of Technology, Free State

FACULTY OF ENGINEERING, BUILT ENVIRONMENT AND INFORMATION TECHNOLOGY

Department of Information Technology

Should you require confirmation or further information regarding this project, please contact me directly using the details below.

Digitally signed by Mpho Mbele
Date: 2026.08.20 13:29:10 +02'00'

**Mpho Mbele**
Departmental Manager
Department of Information Technology
Faculty of Engineering, Built Environment and Information Technology
**E:** mmbele@cut.ac.za | **T:** 051 507 4382

Michael Riese

Private Bag X20539, Bloemfontein, 9300, South Africa
**Tel:** +27 51 5074382 | **Fax:** +27 51 5073655 | **E-mail:** mmbele@cut.ac.za

# Part B - Phase 2: System Architecture and Design

Part B records how the requirements in Part A were turned into a design, and what changed on the way. The design was drawn from the system as actually built (the Flutter app, the ASP.NET app and the shared Supabase database in the GitHub repository), not invented separately, so every diagram below can be checked against running code.

## 4. Requirement and Scope Updates

### 4.1 New Functional Requirements (continues Section 1.7)

| ID | Requirement | Source |
|---|---|---|
| FR-14 | The system shall let a customer check their vehicle's repair status by entering the booking's reference code, without logging in. The customer sees the vehicle, its current phase, and the parts and labour lines on the job, but no staff data and no other booking. Available in the mobile app ("I'm a customer") and on a public web page (/Track). | Academic consultation, 22 Sept 2026 (Change 7) |
| FR-15 | The system shall let staff move a booking through five phases: Book-in & Inspection, Labor in progress, Reassembly, Finished - ready for pickup, Finalized - booked out to customer. Both apps offer the phase change. "Finalized" is set automatically when a booking is closed. | Change 8 |
| FR-16 | The system shall send a push notification to a customer's phone when their booking's phase changes, if the customer opted in on that device. | Change 8 |
| FR-17 | Only an Admin shall be able to finalize a booking, delete a booking, reopen a closed booking, or remove a part or labour line. Mechanics can do everything else. | Group decision, 23 Sept 2026 (Change 9) |
| FR-18 | Deleting a booking shall return every allocated part to stock and remove its labour in a single all-or-nothing operation. | Design, supports NFR-05 (Change 11) |

FR-12 (authenticated staff access) is now met on both clients: mobile gained Supabase login on 28 Sept 2026 (Change 10).

### 4.2 Requirements Change Log - continued (continues Section 1.10)

| # | What changed | Reason |
|---|---|---|
| 7 | **Customer repair-status lookup added (FR-14). Partially reverses Change 5**, which excluded customer-facing real-time job/status tracking. | Raised again and confirmed at the Phase 2 academic consultation with Ms Mbele (22 Sept 2026): customers "don't need to login, even just a screen with updates". The design answers Change 5's two original objections directly. (1) **False expectations:** the customer sees only five coarse phases, never an ETA or a promised completion date, so parts-sourcing delays do not create broken promises. (2) **Extra work for mechanics:** phases are changed at natural milestones with one tap, not as fine-grained progress logging. The Phase 1 exclusion of a customer *login portal* (Change 2) still stands: the lookup uses a one-off random reference code, not an account. **Stakeholder confirmation:** Change 5 was made on Michael Riese's operational preference, so the group took this reversal back to him. **Michael Riese confirmed Change 7** (confirmation reported to the group on 11 October 2026). |
| 8 | Booking phases (FR-15) and phase-change push notifications (FR-16) added. | Makes the FR-14 lookup useful without the customer repeatedly checking. Uses Firebase Cloud Messaging's free tier and a Supabase Edge Function, so the zero-budget constraint (Section 2, Economic) still holds. |
| 9 | Admin and Mechanic roles with admin-only actions (FR-17). | Finalizing, deleting and reopening a booking change money and stock records; Part A's objective that "admin staff track costs" (1.11) needs those actions controlled. Agreed by the group on 23 Sept 2026 and enforced in both apps, with deletes also refused by the database for non-admins. |

| | | |
|---|---|---|
| 10 | Mobile app changed from a role picker to real Supabase Auth login, with the role read from a `profiles` table. | Brings the mobile app in line with FR-12 and NFR-04; the earlier picker let anyone choose "Admin". |
| 11 | Reopen a closed booking (admin only, re-enter password) and atomic delete-with-restock (FR-18) added on the web app. | Mistakes happen after a job is finalized; reopening needs a deliberate, authenticated step. Doing restock and delete in one database transaction keeps stock accurate (NFR-05). |

### 4.3 Refined Scope - amendment (continues Section 1.11)

- **Moved into scope:** a login-free repair-status lookup by reference code showing coarse phases, with optional push notifications (Changes 7 and 8).
- **Still out of scope:** a customer portal or customer accounts; live or granular progress tracking; ETAs or promised completion times; online payment; multi-branch support; accounting integration.
- **Deliverables** are unchanged: Flutter app, ASP.NET app, shared Supabase backend, exportable invoice PDF.

### 4.4 Risk Register - updates (continues Section 3)

| ID | Update | Owner | Status |
|---|---|---|---|
| R-03 | ASP.NET web app risk: the app was built in milestones M0-M8 (login, dashboard, bookings, stock, labour costing, invoice PDF, vehicle history, realtime, testing). | Gary / Wisani | Closed |
| R-02 | Scope growth **materialised**: the consultation added FR-14 and the build added FR-15/16. Handled as R-02 intended, through the change log (Changes 7-8) rather than silently. | Denny | Managed |
| R-10 | **Materialised 28 Sept 2026:** a Firebase service-account key was committed to the GitHub repository. The file was removed and the pattern added to `.gitignore` the same day; because it remains in git history, the key must be revoked and replaced in Google Cloud, exactly as R-10's contingency requires. | Gary | Open - revocation to be confirmed |
| R-14 | **New.** Michael Riese may not accept Change 7, since Change 5 was based on his preference. Mitigation: reconfirm before submission, showing the coarse-phase, no-ETA design. Contingency: disable notifications and keep the lookup staff-facing only. | Denny | Closed - Michael Riese confirmed Change 7 (11 Oct 2026) |
| R-15 | **New.** Two sources of truth for "who is an admin": the web app's configured admin list and the database `profiles` table. If they drift, a user could be admin in one app and not the other. Mitigation: keep one admin account list and check both when staff change. | Wisani | Open |
| R-17 | **New, found 7 Oct 2026.** The staff function that changes a booking's phase could be called **without logging in** using the public app key, because Postgres lets everyone execute new functions unless that is revoked. Someone with a booking's internal id could change its phase (sending the customer a notification) and read the booking row. The delete function was already protected. Fix: migration 0008 revokes anonymous access to all staff functions; the customer lookup stays public by design. | Gary | Open - 0008 to be run |
| R-16 | **New.** Labour is recorded in two tables: mobile uses `booking_labor`, web uses `jobs`. Invoice totals can differ depending on which app produced them. Mitigation: agree one labour table before Phase 3 and migrate the other. | Gary | Open |

# 5. System Design

## 5.1 System Architecture (Activity 3.1)

Spanner is one integrated system with two staff clients and one shared backend. Both the Flutter app and the ASP.NET app talk to Supabase directly; there is no separate custom API layer. Supabase provides authentication, the Postgres database protected by Row Level Security, the REST (PostgREST) interface, database functions for multi-step operations, and Realtime updates to the web dashboard. The only server-side code outside the two apps is one Supabase Edge Function that sends phase-change notifications through Firebase Cloud Messaging.

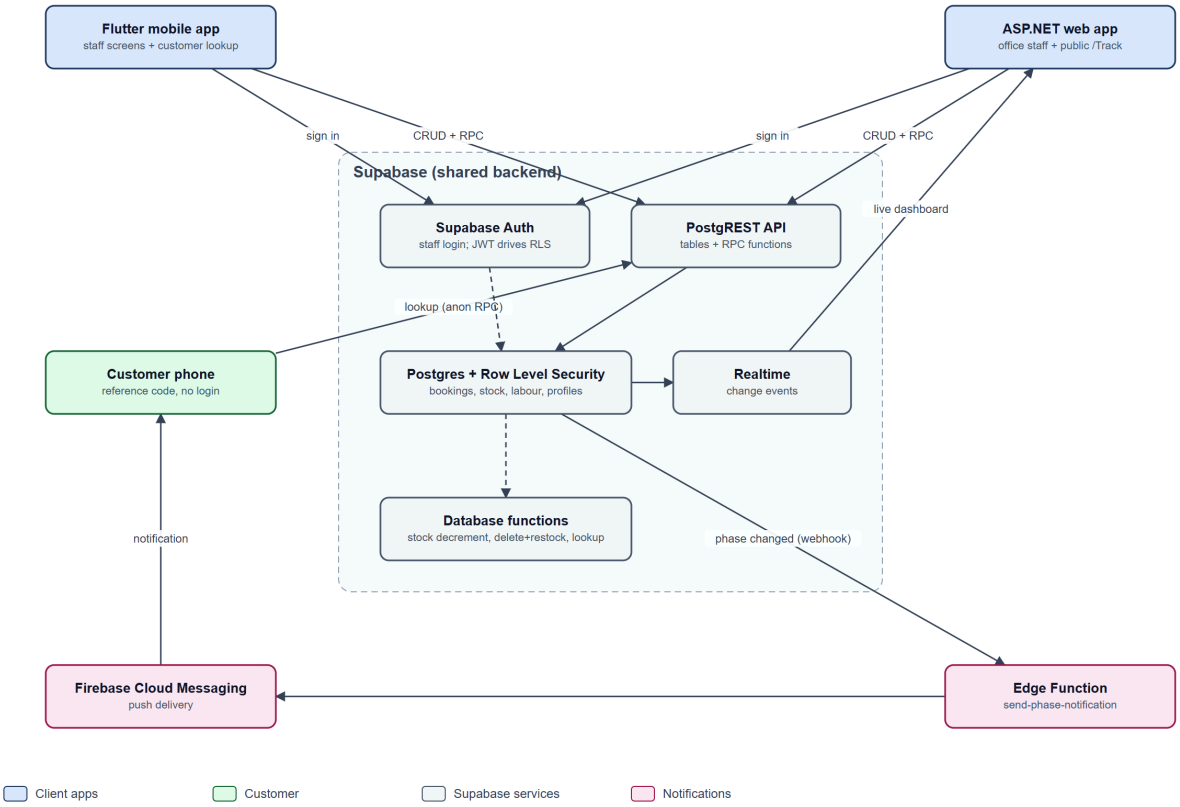



Figure 1 - System architecture (editable source: architecture.drawio)

| Component | Role |
|---|---|
| Flutter mobile app | Workshop-floor client for staff (bookings, phases, parts, labour, stock, invoice export) and the login-free customer lookup screen. |
| ASP.NET web app | Office client for staff: dashboard with live updates, bookings, stock, invoices, vehicle history, admin-only actions. |
| Supabase | Single source of truth: Auth, Postgres + Row Level Security, PostgREST, database functions (e.g. atomic stock decrement, delete-with-restock, reference-code lookup), Realtime. |
| Edge Function + Firebase Cloud Messaging | Triggered by a database webhook when a booking's phase changes; sends a push notification to subscribed customer devices. |

**Key data flows.** (1) A booking created on either app is immediately visible on the other, because both read and write the same tables. (2) Adding a part decrements stock inside one database function, so a failure cannot lose stock. (3) Admin-only actions are hidden in both apps and rejected server-side; deletes are also refused by the database for non-admins. (4) The customer lookup is the only unauthenticated path: it calls a narrowly scoped database function, and anonymous users have no direct read access to any table.

## 5.2 Use Case Diagram (Activity 3.2)

The use case diagram follows the structure agreed at the 22 Sept consultation: Admin on the left and Mechanic on the right, each shared use case connected to both, and admin-only use cases connected to Admin alone. Login includes validation with Supabase Auth and a dashboard, which is a landing page, not a hub other use cases return to. The Customer is a separate actor with no login.

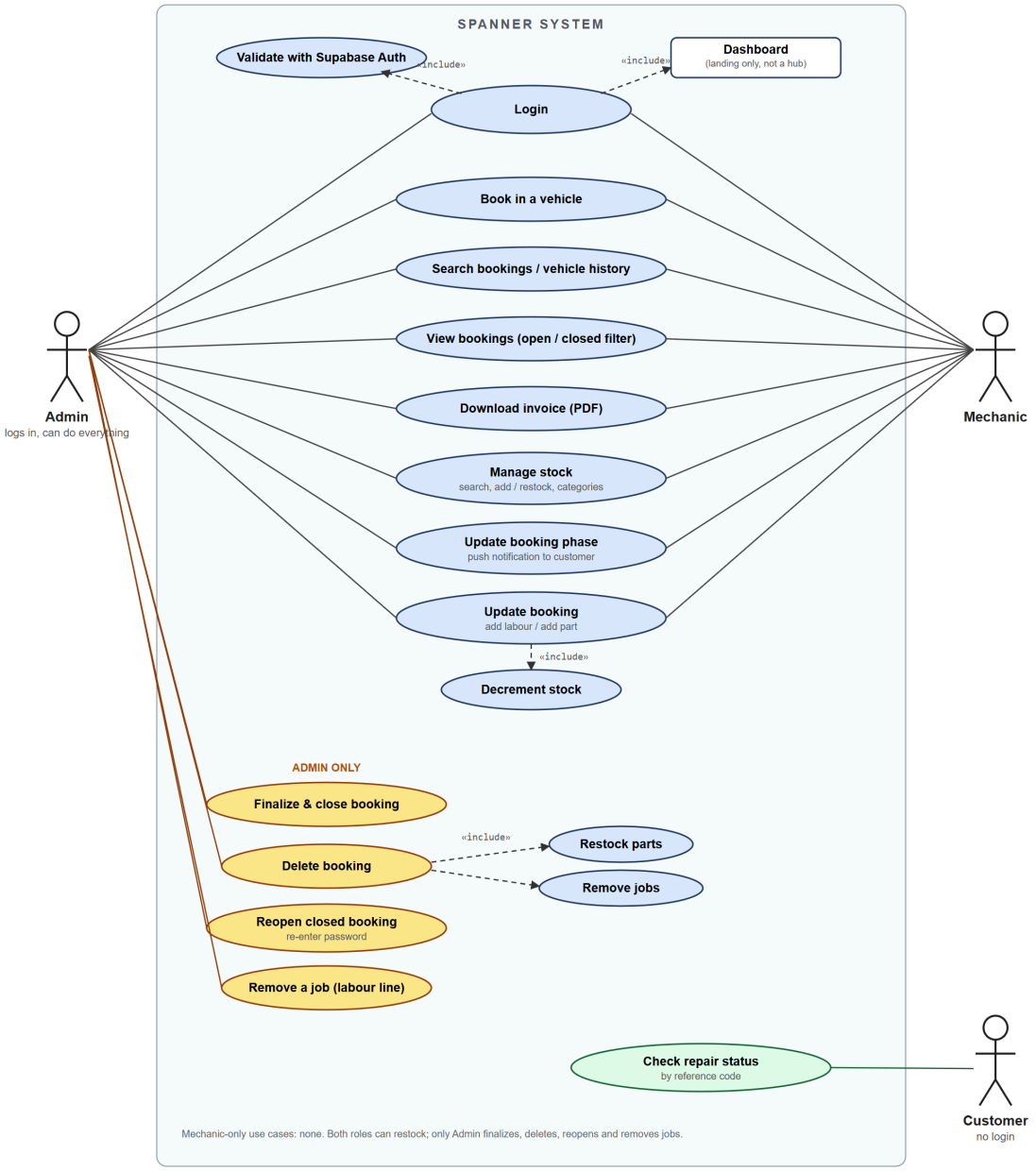



Figure 2 - Use case diagram

| Use case | Mechanic | Admin |
|---|---|---|
| Login; book in a vehicle; search bookings / vehicle history; view bookings (open/closed filter); download invoice; manage stock (incl. restock); update booking (add labour/parts); update booking phase | Yes | Yes |
| Finalize & close booking; delete booking (includes restock parts, remove jobs); reopen closed booking (re-enter password); remove a part or labour line | No | Yes |
| Check repair status by reference code (Customer, no login) | Customer only | |

## 5.3 Sequence Diagrams (Activity 3.2)

Every sequence diagram starts with authentication and then shows the work, and uses **alt** frames (one branch or the other) for decisions. Five diagrams are included: at least one for each client, as the brief requires.

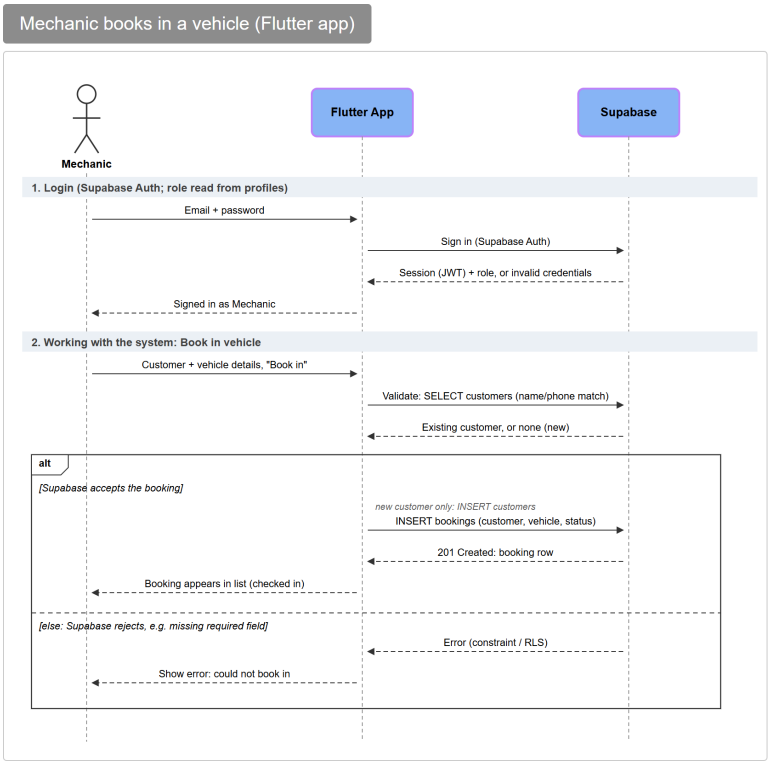



Figure 3 - Mechanic books in a vehicle (Flutter app)

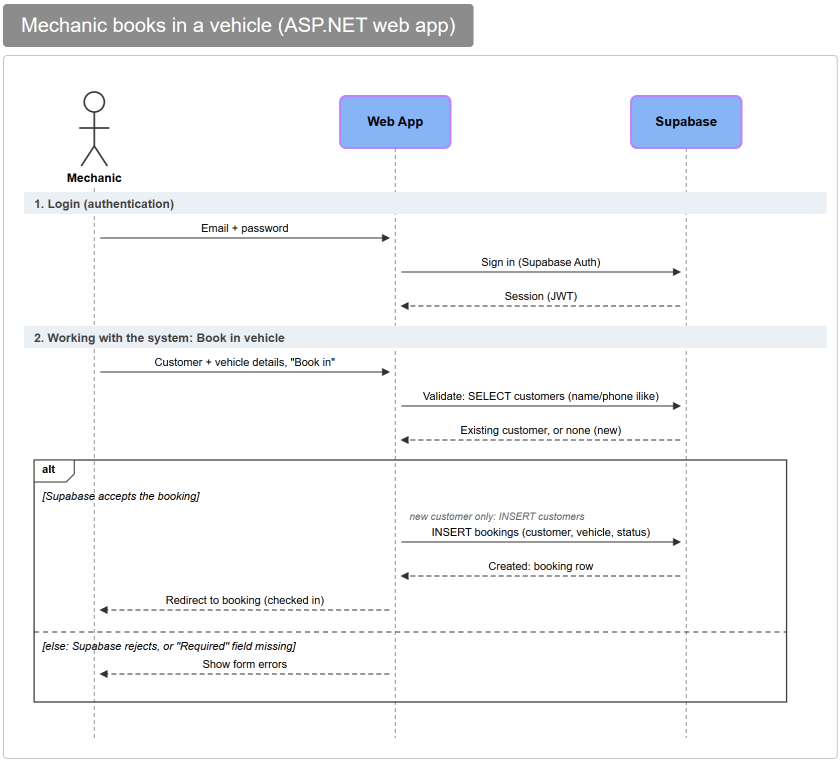



Figure 4 - Mechanic books in a vehicle (ASP.NET web app)

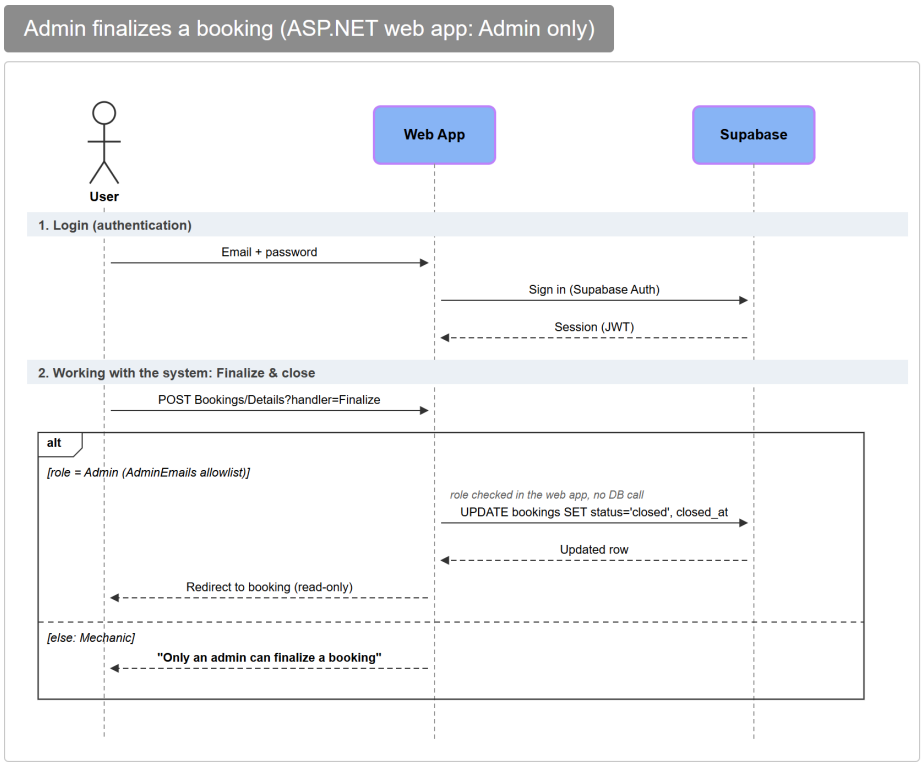



Figure 5 - Admin finalizes a booking (the admin check happens in the web app, not as a database call)

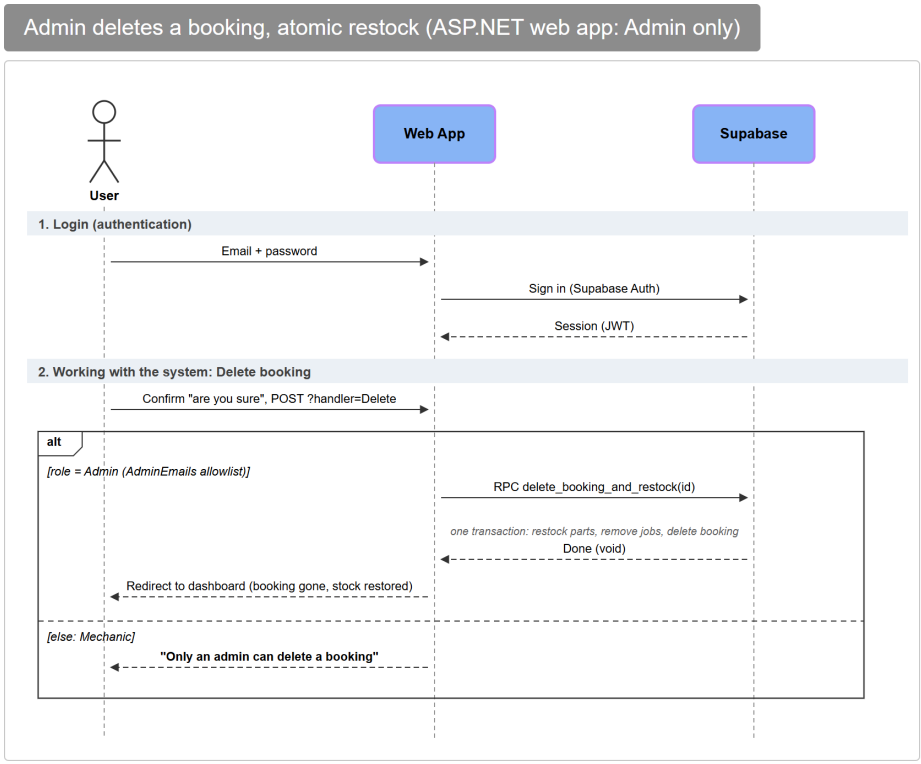



Figure 6 - Admin deletes a booking with atomic restock

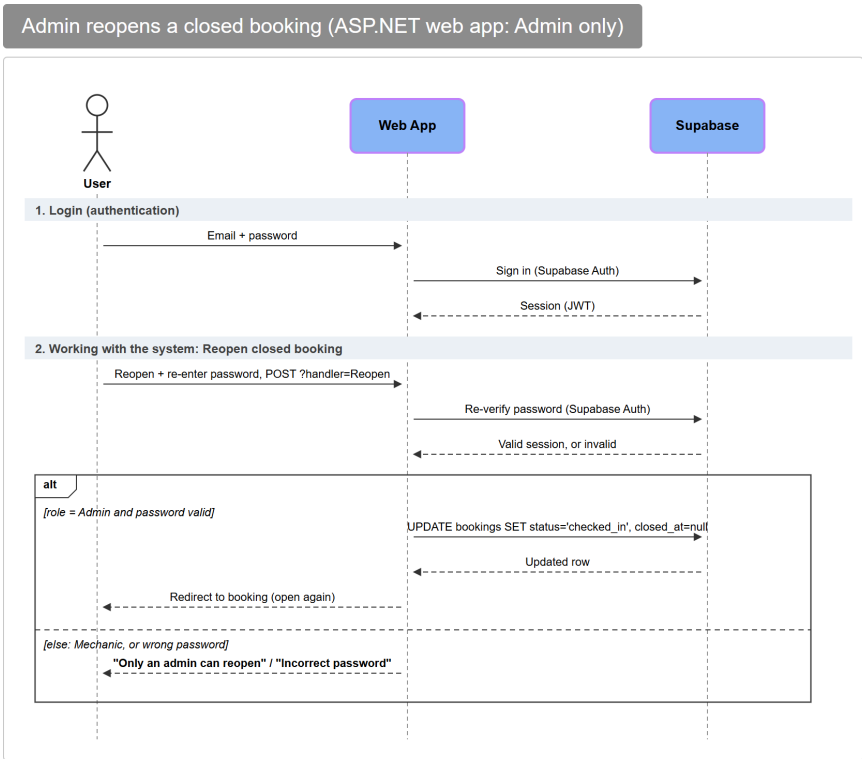



Figure 7 - Admin reopens a closed booking (password re-verified)

## 5.4 Entity Relationship Diagram and Database Design (Activity 3.2)

The ERD reflects the live Supabase schema. `bookings` is the centre of the model: it links a customer, the parts used (`booking_parts`, which reference `stock_items`), labour (`jobs` on web, `booking_labor` on mobile, see R-16), and customer push subscriptions. Every stock change is also written to `stock_movements` as an audit trail (NFR-05). Staff roles live in `profiles`, one row per authenticated user.

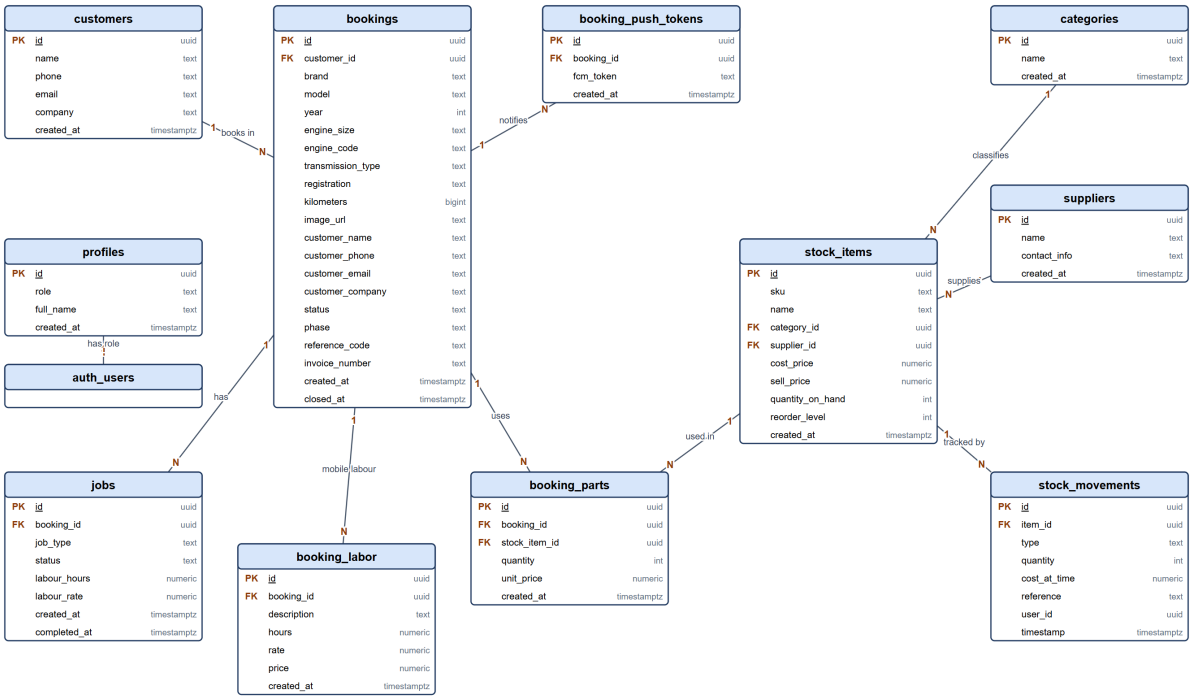



Figure 8 - Entity relationship diagram: primary and foreign keys, and 1/N cardinality on every relationship (editable source: erd.drawio)

| Rule | How the database enforces it |
|---|---|
| Stock can never go negative through a part allocation | Single database function decrements and records the movement in one transaction. |
| Deleting a booking restores stock exactly | One function restocks, logs, removes labour and parts, then the booking (FR-18); non-admins refused. |
| Customers see only their own booking | Random, unique `reference_code`; lookup through a security-definer function returning a fixed, minimal shape. |
| Only staff read or change shop data | Row Level Security on every table (NFR-04). |

## 5.5 Flutter Interface Design (Activity 3.3)

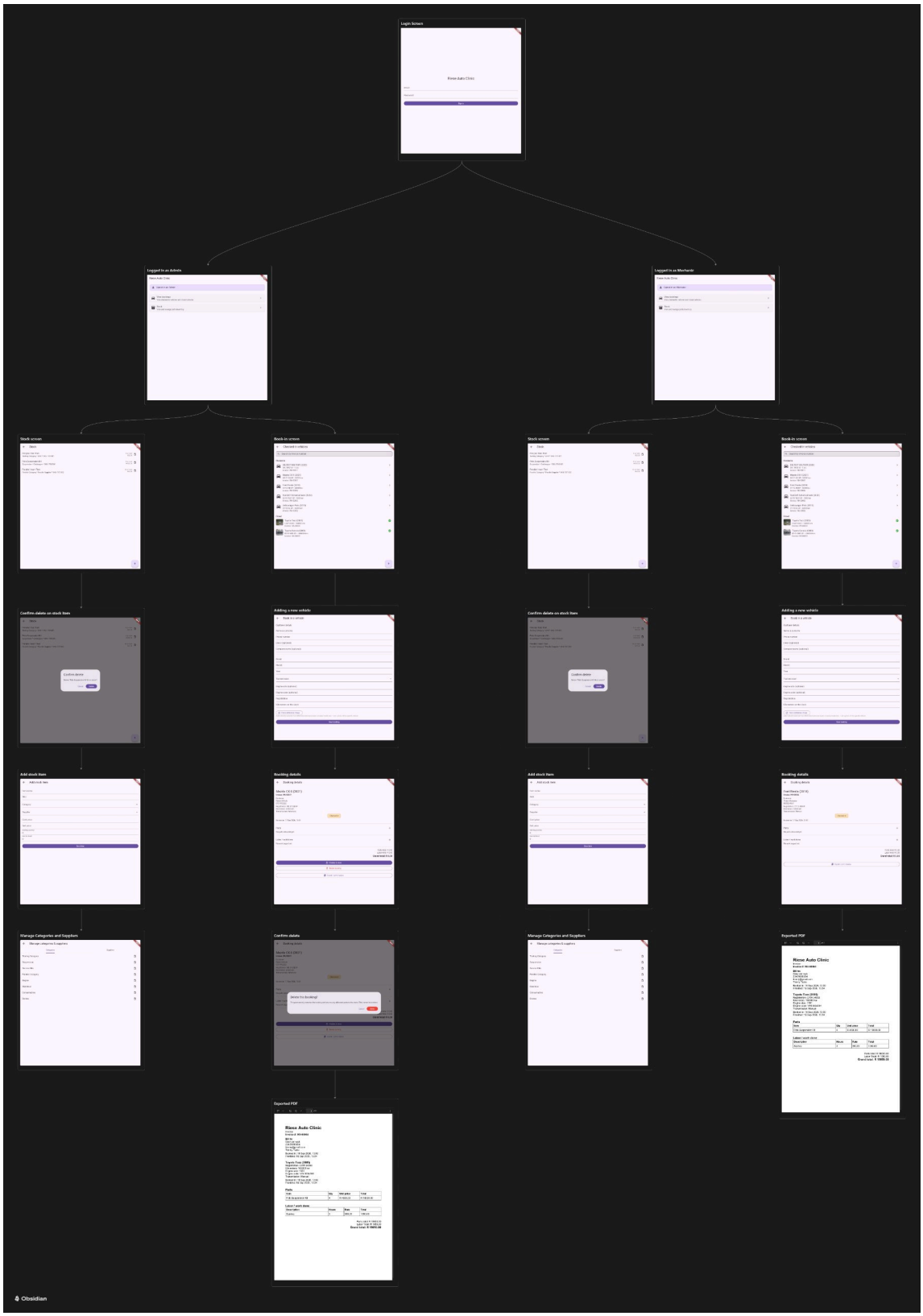



Figure 9 - Flutter wireframe flow by Gary van Wyk (login, admin and mechanic branches, stock, book-in, booking details, confirmations, exported invoice). Drawn before the customer screens were added on 28 Sept.

### 5.5.0 Flutter screen mockups (11 Oct)

Gary turned the wireframe flow into high-fidelity mockups of the five main screens. They use the agreed phase names (Book-in & inspection, Labor in progress, Reassembly, Finished - ready for pickup) and the orange and green accents that the client later confirmed. The booking detail screen (C) shows every value a mechanic captures: customer, vehicle, phase, parts and labour lines with running totals, and the admin-only "Finalize & close". Each value maps to a column in the ERD (Fig. 8).

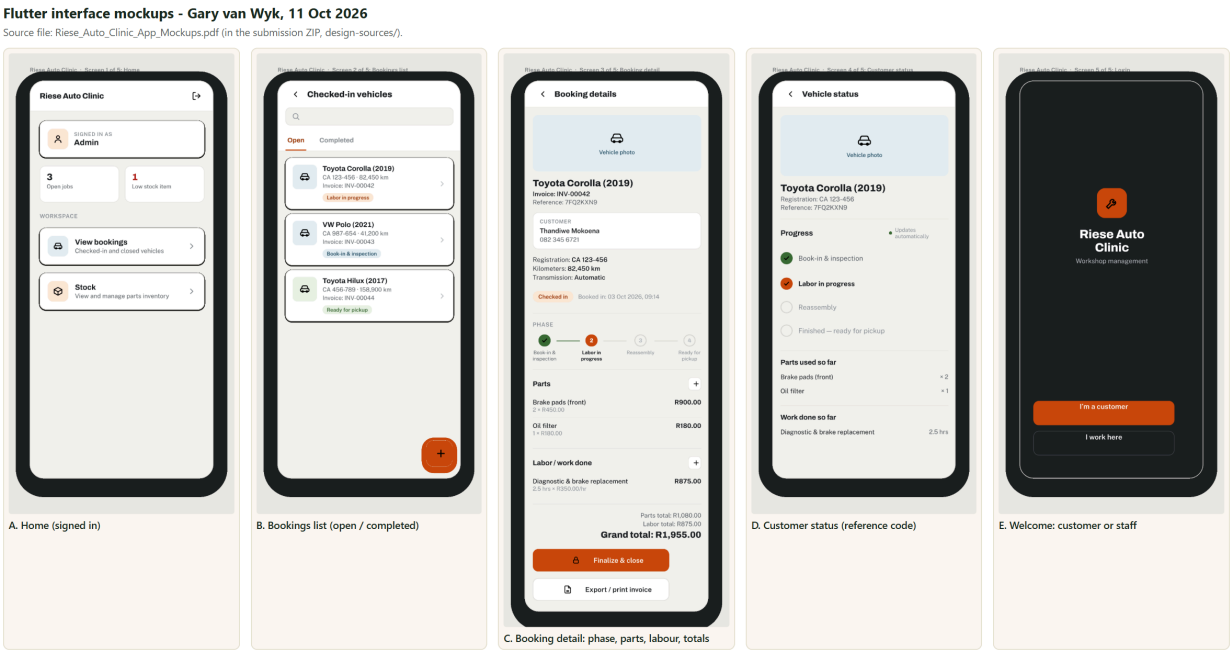



Figure 9a - Flutter mockups by Gary van Wyk: A home, B bookings list, C booking detail, D customer status, E welcome (source: Riese_Auto_Clinic_App_Mockups.pdf). Sample data.

So that every interface design has an editable source in a standard design tool, the mobile screens were redrawn in draw.io in the client's colours, with the navigation between them (Fig. 9b). It is page 10 of `spanner-all-designs.drawio`.

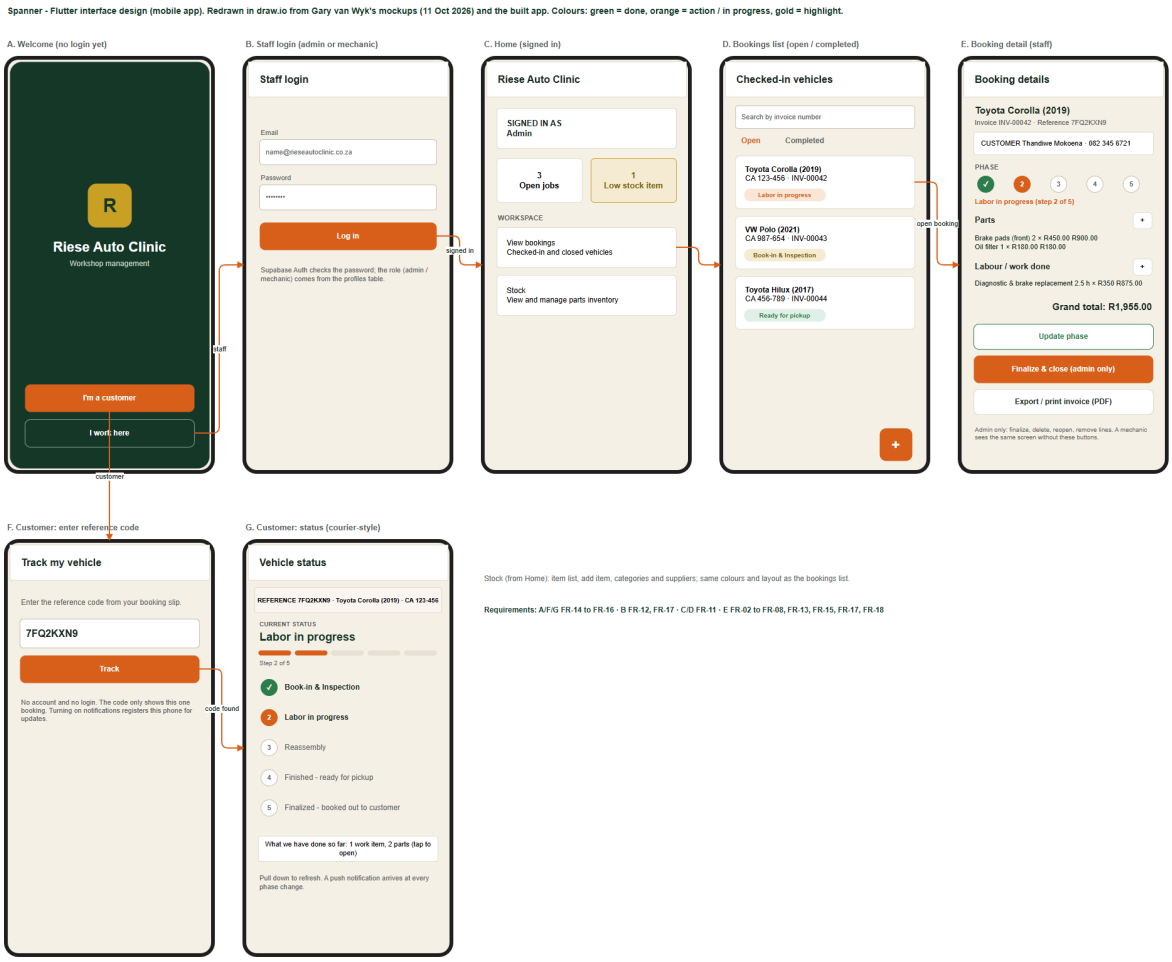



Figure 9b - Flutter interface design in draw.io: A welcome, B staff login, C home, D bookings list, E booking detail, F customer code entry, G customer status (editable source: flutter-screens.drawio).

### 5.5.1 Customer tracking screen (mobile)

The customer screen is designed like a courier tracking page (for example PUDO or The Courier Guy), because customers already understand that pattern: a strip with their reference code and vehicle, one large "current status", a five-step progress bar and a timeline of the stages. The customer reaches it from the welcome screen ("I'm a customer"), enters the reference code from their booking slip, and can pull down to refresh; push notifications announce each change. The screenshots are the real Flutter screen rendered from the project code with sample data. It compiles with no analyzer errors and is with Gary for review before it replaces the current screen.

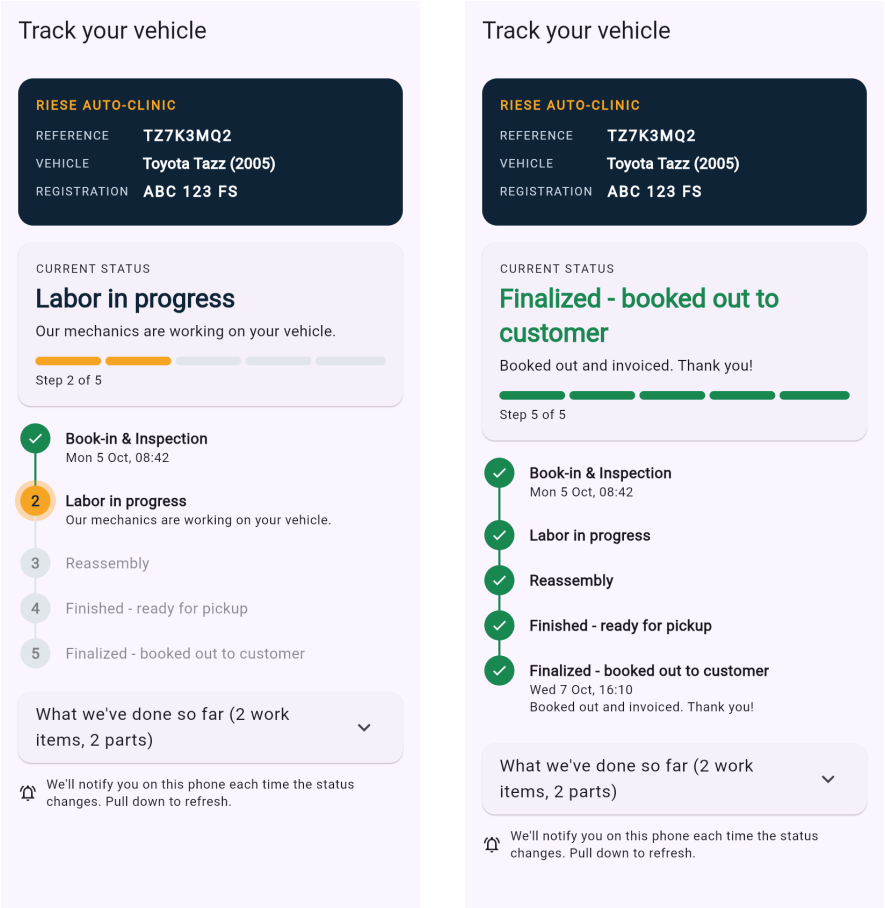



Figure 11 - Mobile customer tracking: work in progress (left) and booked out (right). Sample data; the registration is fictional.

## 5.6 ASP.NET Interface Design (Activity 3.3)

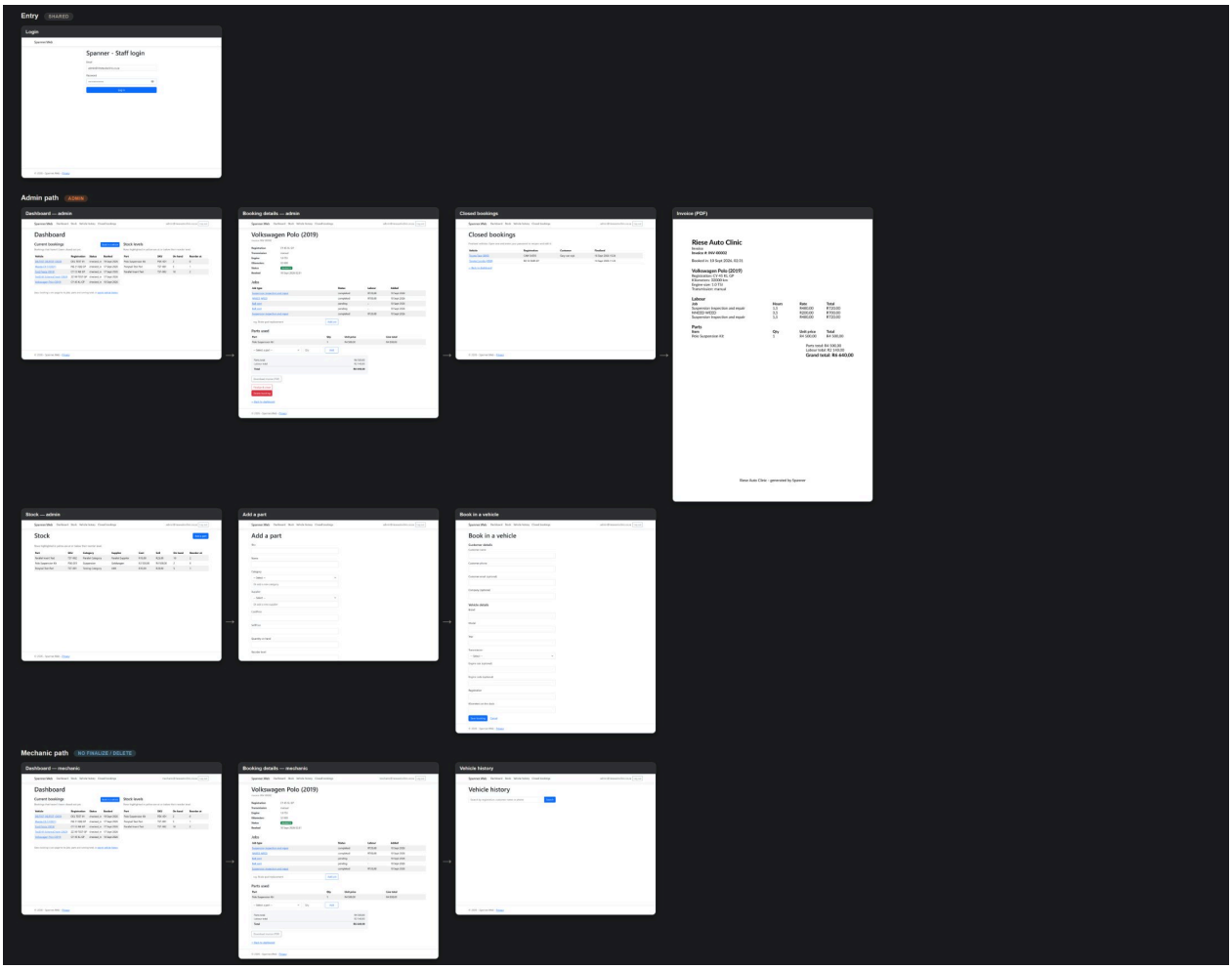



Figure 10 - ASP.NET web app screens captured from the running system, 20 Sept 2026: shared login, admin path (dashboard, booking details, closed bookings, invoice PDF, stock, add a part, book in a vehicle) and mechanic path (no finalize or delete).

Since these captures the booking page has also gained a **phase selector** (both roles, FR-15) and the admin-only Reopen.

### 5.6.0 Web design and the client's colours (7-11 Oct)

Denny designed the web interface as a prototype (Fig. 10a): a dark sidebar for navigation, a "Today" page with stat cards, a bookings table, the book-in form, stock with low-stock flags, and a phase progress bar on the booking. When we asked about colours in October, the client chose **gold and green, or gold, green and orange**. The built web app now uses Denny's layout in those colours:

| Colour | Used for | Why |
|---|---|---|
| Dark green #173A2C | Sidebar, headers, login card | Brand colour; high contrast with white text |
| Gold #C9A227 | Logo, active menu item, highlights, low-stock rows | Client's accent colour; draws the eye without meaning "error" |
| Orange #D9601A | Action buttons, the current phase | "Something to do / in progress", as in Gary's mobile mockups |
| Green #2E7D4F | Completed phases, "ready for pickup" | "Done", the same meaning in both apps |

Two parts of Denny's prototype were not adopted, so that the design stays aligned with the approved requirements: its phase names (Inspection, Repair in progress, Quality check, Ready for collection) were replaced with the five agreed phases that the database, the mobile app and the push notifications already use (FR-15), and its SMS/WhatsApp message step was not built because customers are notified by push notification (FR-16).

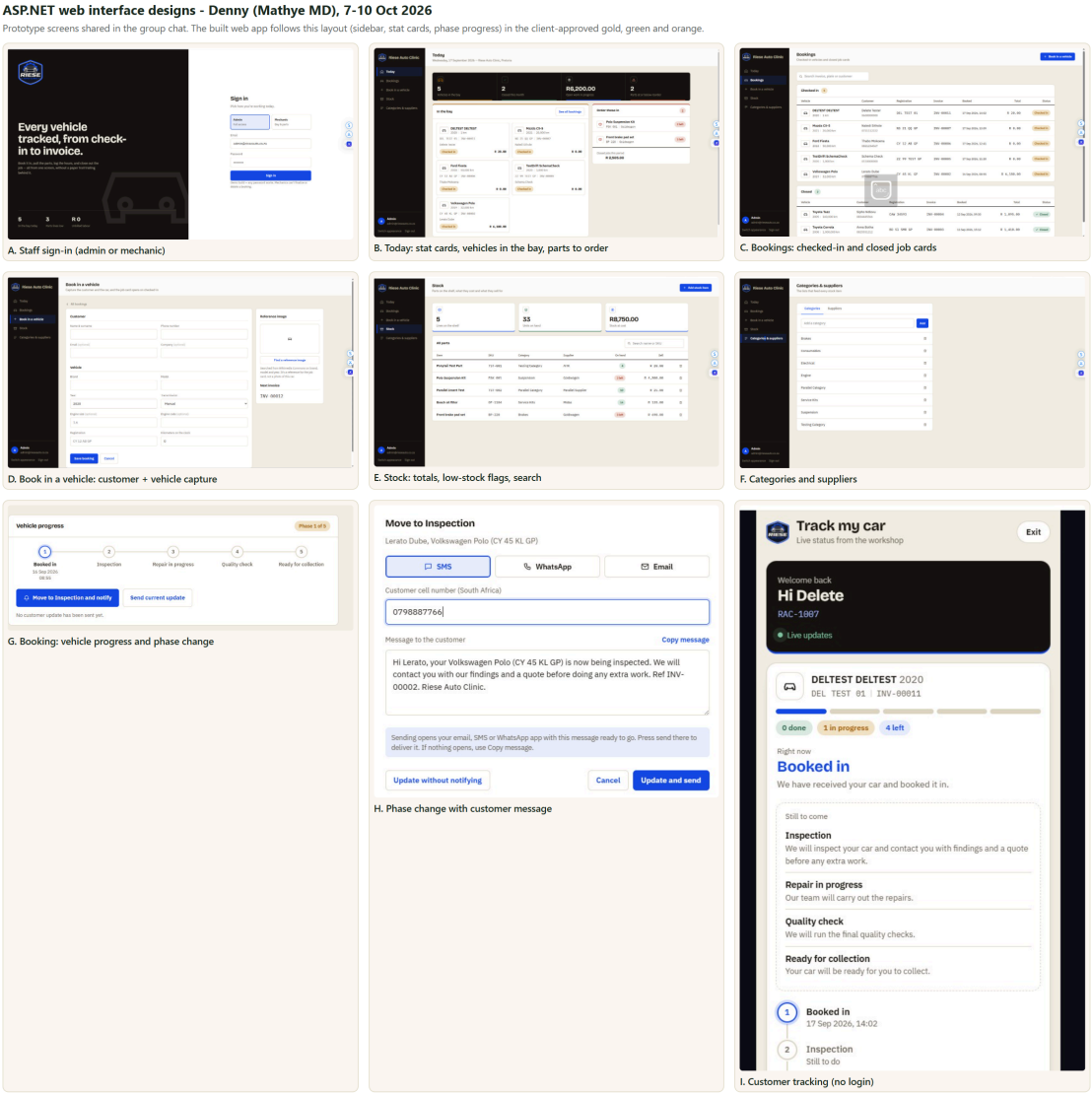



Figure 10a - ASP.NET interface designs by Denny (Mathye MD), 7-10 Oct 2026. Demo data from the prototype.

The web pages were also redrawn in draw.io from Denny's layout and the built app (Fig. 10b), page 11 of `spanner-all-designs.drawio`.

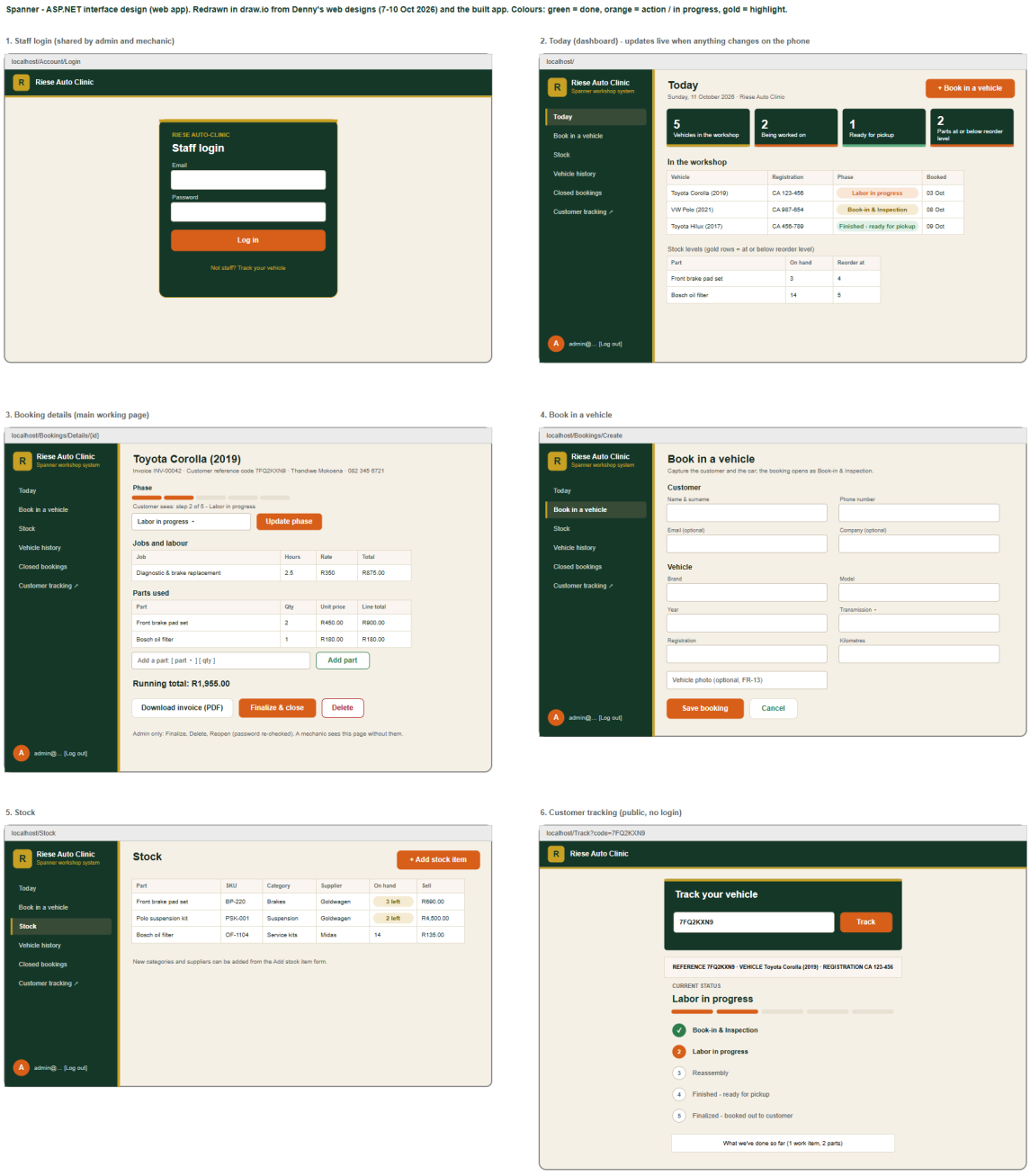



Figure 10b - ASP.NET interface design in draw.io: 1 staff login, 2 Today dashboard, 3 booking details, 4 book in a vehicle, 5 stock, 6 customer tracking (editable source: aspnet-screens.drawio).

### 5.6.1 Customer tracking page (web, /Track)

A public page that needs no login, linked from the staff login ("Not staff? Track your vehicle"). It uses the same layout, wording and five stages as the mobile screen and the push notification, so the customer sees one consistent story. It calls only the reference-code lookup function. It was built and tested against the live database (empty, not-found and invalid-code states); the screenshots show the found state with sample data.

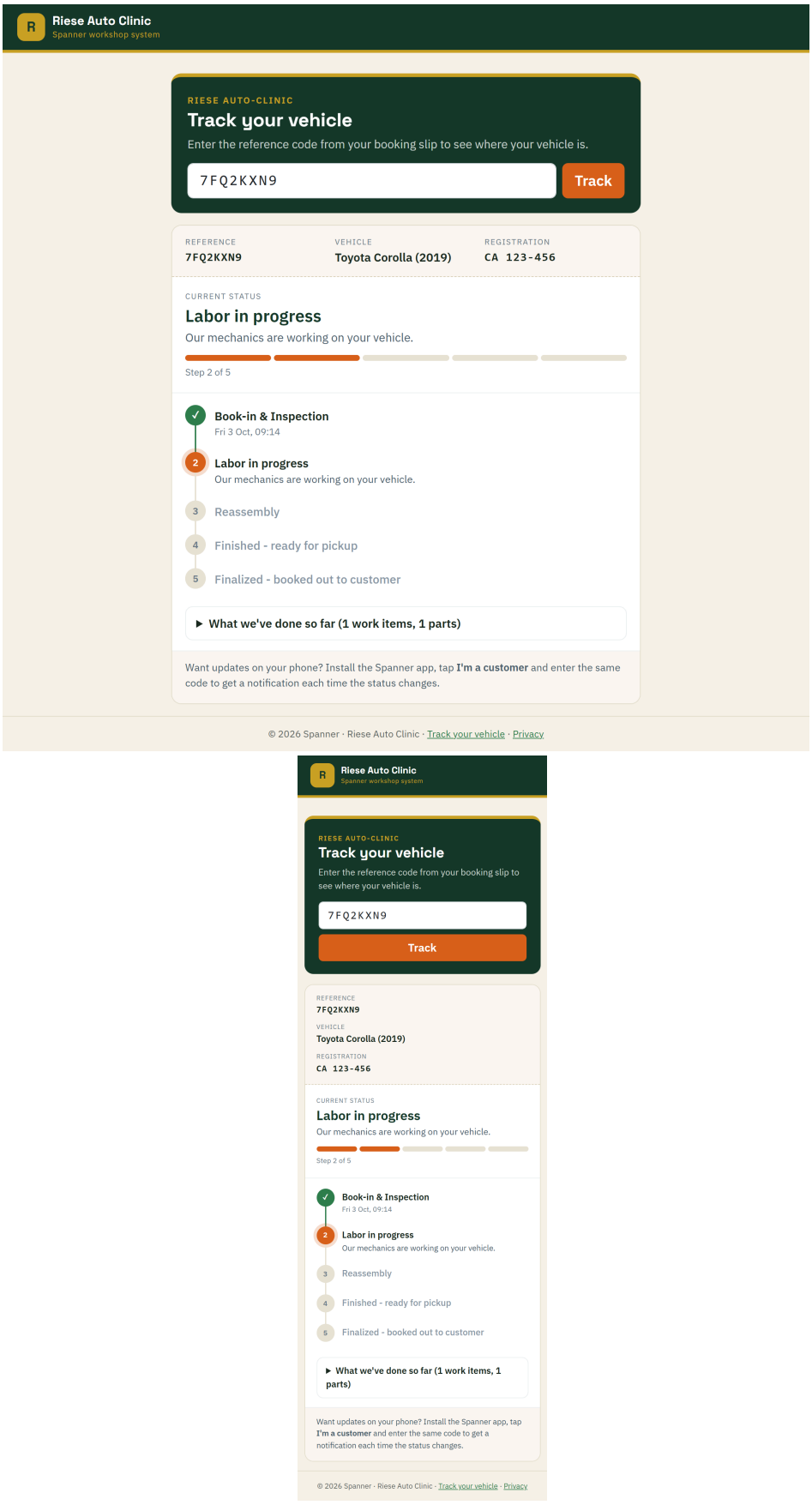



Figure 12 - Web customer tracking page in the client colours, on a laptop (left) and a phone browser (right). Captured from the running app on 11 Oct 2026 with the identifying details replaced by sample data.

## 5.7 Navigation Flow (Activity 3.3)

How each kind of user moves between the screens of both apps. Customers (green) never pass through a login; staff screens (blue) are the same for both roles; admin-only actions (yellow) sit on the booking details screen of each app.

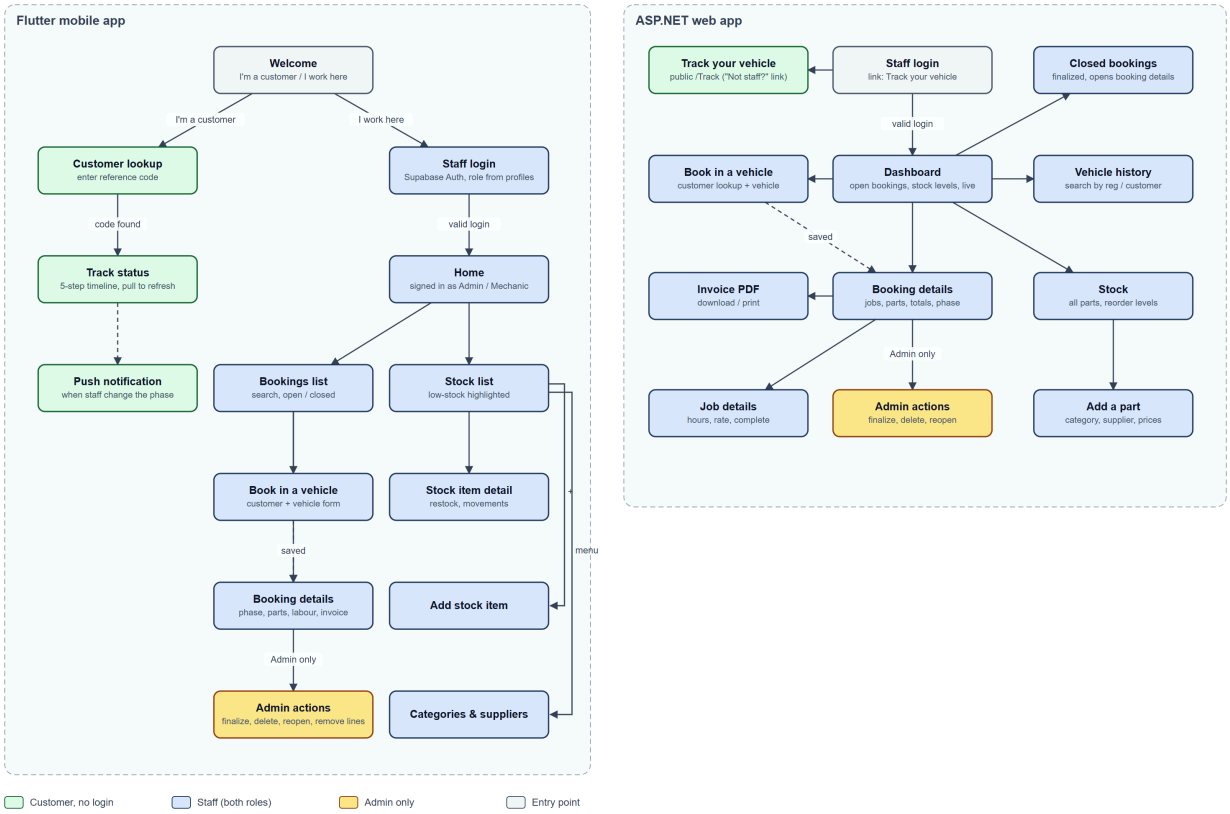



Figure 13 - Navigation flow, Flutter and ASP.NET (editable source: navigation-flow.drawio), drawn from the real routes in both code bases.

# 6. Design Alignment, Sources and Open Items

## 6.1 Design Alignment - requirement to design traceability

> Draft compiled from the design above for Denny's review (task U3-13).

| Req. | Use case | Sequence | Data (ERD) | Interface |
|---|---|---|---|---|
| FR-01 | Book in a vehicle | Fig. 3, 4 | customers, bookings | Book-in screens (Fig. 9, 10) |
| FR-02, FR-05, FR-06 | Update booking (add labour) | - | jobs / booking_labor | Booking details |
| FR-03, FR-10 | Manage stock | - | stock_items, categories, suppliers | Stock, add a part |
| FR-04 | Update booking (add part) incl. Decrement stock | - | booking_parts, stock_movements | Booking details |
| FR-07, FR-08 | Download invoice | - | bookings, booking_parts, labour | Invoice PDF |
| FR-09 | (all, via shared backend) | - | all tables | Web dashboard live refresh |
| FR-11 | Search bookings / vehicle history | - | bookings | Vehicle history |
| FR-12 | Login (includes Validate with Supabase Auth) | All (step 1) | profiles | Login screens |
| FR-13 | Book in a vehicle | - | bookings.image_url | Booking details |
| FR-14 | Check repair status (Customer) | - | bookings.reference_code | Customer tracking, mobile and web /Track (Fig. 11, 12) |
| FR-15, FR-16 | Update booking phase | - | bookings.phase, booking_push_tokens | Phase selector on booking details (both apps) |
| FR-17 | Admin-only use cases | Fig. 5, 6, 7 | profiles | Admin vs mechanic paths (Fig. 10) |
| FR-18 | Delete booking (includes restock) | Fig. 6 | booking_parts, stock_movements | Booking details |

## 6.2 Editable Design Sources (Appendix F)

**All designs in one file:** `spanner-all-designs.drawio` has one page per design (architecture, use case, five sequence diagrams, ERD, navigation flow, Flutter screens, ASP.NET screens). It is in the ZIP and opens directly from GitHub, with no account, at:
`https://app.diagrams.net/#Uhttps%3A%2F%2Fraw.githubusercontent.com%2FBlackZ3ro%2FSpanner-Riese-Auto-Clinic%2Frefs%2Fheads%2Fphase2%2Fdesign-docs%2Fproject-docs%2Fphase-2%2Fdiagrams%2Feditable%2Fspanner-all-designs.drawio`

The brief requires a working link to, or the editable file of, every design; exported images alone do not count. draw.io (diagrams.net) is free and works online or offline, so its editable files are included in the submission ZIP under `design-sources/` and in the GitHub repository under `project-docs/phase-2/diagrams/editable/`. Open any of them at app.diagrams.net (File, Open from, Device).

| Design | Tool | Editable file (ZIP and GitHub) / link |
|---|---|---|
| System architecture | draw.io | `architecture.drawio` |

| | | |
|---|---|---|
| Use case diagram | draw.io; Lucidchart | `usecase.drawio` `https://lucid.app/lucidchart/fb4ebb76-2892-4f1a-888f-067c37aca98d/view` |
| Sequence diagrams (5) | draw.io; Lucidchart | `seq-1-book-in-vehicle-flutter.drawio, seq-1b-book-in-vehicle-web.drawio, seq-2-admin-finalize-web.drawio, seq-3-admin-delete-web.drawio, seq-4-admin-reopen-web.drawio` `https://lucid.app/lucidchart/285c1247-0f85-4e60-879c-90f48a0c91f6/view` |
| ERD | draw.io | `erd.drawio` |
| Navigation flow | draw.io | `navigation-flow.drawio` |
| Flutter wireframes | Obsidian Canvas | *Pending: Gary to add the .canvas file to the ZIP (U3-10)* |
| Flutter mockups (Fig. 9a) | Gary's mockup tool (exported PDF) | `Riese_Auto_Clinic_App_Mockups.pdf` *(editable source link: Gary)* |
| Flutter screens (Fig. 9b) | draw.io | `flutter-screens.drawio (page 10 of spanner-all-designs.drawio)` |
| ASP.NET screens (Fig. 10b) | draw.io | `aspnet-screens.drawio (page 11 of spanner-all-designs.drawio)` |
| ASP.NET designs (Fig. 10a) | Denny's HTML prototype | `denny-web-designs.png` *(prototype source or link: Denny)* |
| Customer tracking screens | Flutter / ASP.NET (code) | `GitHub: web/Pages/Track; mobile branch proposal/mobile-customer-tracking` |
| ASP.NET screens | Running application | `GitHub: web/Pages` |
| Microsoft Project schedule | Microsoft Project | `TheZealotsSchedule.mpp (Khumo, 8 Oct); TheZealotsSchedule (Phase 2 update 11 Oct).xml` |

### 6.3 Microsoft Project schedule

Khumo's schedule (8 Oct) holds Phase 1 and the Unit 3 design activities with owners and finish-to-start dependencies. On 11 Oct it was brought up to the actual state: every finished task marked 100%, and three sections added - development, integration and testing (Supabase, ASP.NET, Flutter, the customer tracking page, end-to-end testing and the redesign), the submission and defence (12 and 16 Oct), and Phase 3 to 2 November. The status date is 11 October (red line).

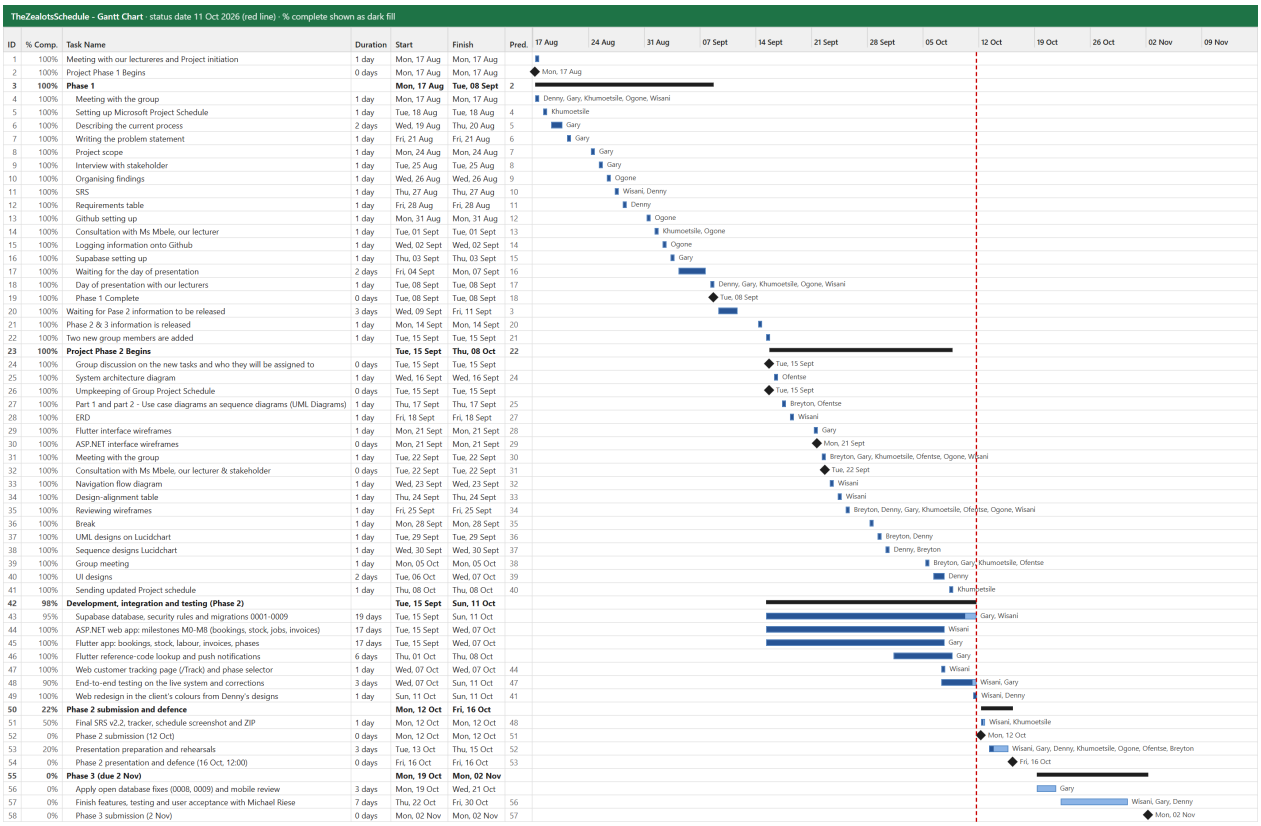



Figure 16 - Project schedule, Gantt view at 11 Oct 2026 (TheZealotsSchedule, Phase 2 update). % complete is the dark fill; diamonds are milestones.

> Before submission: Khumo opens the .xml in Microsoft Project, saves it as TheZealotsSchedule.mpp and replaces Figure 16 with a screenshot of the Gantt Chart view.

## 6.4 Items outstanding before submission (12 October 2026)

| Item | Owner |
|---|---|
| Open the updated schedule (.xml) in Microsoft Project, save as .mpp, screenshot for Fig. 16 | Khumo |
| Run migrations 0009 then 0004 then 0008 (R-17; fixes "add part" properly); confirm the Firebase key was revoked (R-10); review the mobile tracking proposal | Gary |
| Editable source or share link for the Flutter mockups (Fig. 9a) and the .canvas wireframes | Gary |
| Prototype source or share link for the web designs (Fig. 10a) | Denny |
| Lucid links: set sharing to "anyone with the link can view" | Breyton |